Roll Control Design and Simulation of a Canard-controlled Sounding Rocket

Waterloo Rocketry 2025 – Aurora

Finn Breuer [1] Joe Dolina [2]

April 15, 2025

[1]Hamburg University of Technology, *finn.breuer@tuhh.de, fjbreuer@uwaterloo.ca*
[2]University of Waterloo, *j2dolina@uwaterloo.ca*

Contents

Introduction **1**
Team History . . . 1
Aurora rocket . . . 2
Development Overview . . . 2

I Controller & Estimator **4**
Finn Breuer

1 Modelling . . . 4
1.1 Kinematics and Quaternions . . . 5
1.2 Sensor measurements . . . 8
1.3 Atmosphere and air data . . . 11
1.4 Aerodynamics . . . 12
1.5 Actuators . . . 16
1.6 Estimator model equations . . . 17
1.7 Controller model equations . . . 19
2 Controller . . . 21
2.1 Controller roll model . . . 21
2.2 Control law and tuning . . . 22
2.3 Optimal control theory . . . 23
2.4 Gain scheduling . . . 23
2.5 Response analysis . . . 24
3 Estimator . . . 26
3.1 EKF algorithm . . . 26
3.2 Optimal estimation theory . . . 28
3.3 Filter tuning . . . 29
3.4 Response analysis . . . 29
3.5 Implementation notes . . . 30
3.6 Pad filter . . . 31
3.7 Hard calibration . . . 32

II Flight Simulation **33**
Joe Dolina

4 Background . . . 33
4.1 Borealis Airbrakes . . . 33
4.2 Project Jupiter . . . 34
5 Theory . . . 34
6 Implementation . . . 35
6.1 Simulation Setup . . . 35
6.2 Top Level Plant . . . 36
6.3 Actuator . . . 36
6.4 Sensors . . . 37

6.5 Rocket Flight . . . 38
6.6 Forces and Moments . . . 39
7 Plant simulation results . . . 42
7.1 Comparison with OpenRocket . . . 42

III Results **44**
8 Simulation . . . 44
8.1 Estimator in the loop . . . 44
8.2 Controller in the loop . . . 44
8.3 Full closed-loop . . . 44
9 Hardware in the Loop . . . 44
10 Testflight . . . 44
11 Competition flight . . . 44

References **45**

A Design appendix **47**
A.1 Numerical methods . . . 47
A.1.1 Numerical Jacobian computation . . . 47
A.1.2 Numerical Integration methods . . . 48
A.2 Jacobians . . . 49
A.2.1 Kinematics and Quaternions . . . 49
A.2.2 Sensor measurements . . . 52
A.2.3 Atmosphere and air data . . . 52
A.2.4 Aerodynamics . . . 52
A.2.5 Actuator . . . 53
A.2.6 Airfoil . . . 53

List of Figures

0.1 Aurora wireframe from OpenRocket 2

1.1 Diagram of the body-fixed frame f^B 5
1.2 Diagram of attitude frames 6
1.3 2D Visualization of the measured magnetic field. 10
1.4 US standard atmosphere model 12
1.5 Diagram of the aerodynamic angles 13
1.6 Swept delta airfoil theory 14
1.7 Diagram of the canard angle 15
1.8 Mechanical design of the actuator 16
2.1 Block diagram of the control loop 21
2.2 Scheduling surfaces along dynamic pressure and canard coefficient 24
2.3 Closed-loop step responses 25
2.4 Closed-loop frequency response 25
3.1 Closed-loop estimation responses 30

4.1 Airbrakes control loop structure 33
6.1 Rocket plant model interface 36
6.2 Canards Actuator Dynamics 36
6.3 Sensor simulation 37
6.4 Standard IMU component instance 37
6.5 Rocket flight system 38
6.6 External forces and moments 40
6.7 Canard roll torque, roll forcing component 41
6.8 Lift component subsystem 41
7.1 Simulation compared with OpenRocket 42

A.2.1 Canard airfoil approximation 54

Introduction

Team History

Waterloo Rocketry [1] is a student team representing the University of Waterloo (Ontario, Canada), in the 2025 Launch Canada Challenge (LC) [2] with a high-altitude liquid bipropellant rocket. The primary objective of the team is to provide students with practical engineering challenges, while maintaining an atmosphere of learning and collaboration which is key to ensure team growth, continuity, and year-over-year competitiveness. Our continued success is a testament to the significant time past and present members have dedicated to the team, as well as the support and resources provided by the University of Waterloo, and the team's advisors sharing valuable insights as the team develops more complex and sophisticated systems.

State estimation and control projects have been a topic of discussion on the team for the past decade. The payload that flew on the team's 2023 competition rocket, *Leviathan of the Sky*, was essentially a state estimation prototype which relied on mostly existing avionics and sensors. Due to both hardware and software issues the project produced no useful flight data, although the rocket launched successfully[1] in the Intercollegiate Rocket Engineering Competition (IREC, at Spaceport America, New Mexico).

It became clear that an independant payload was not a suitable platform for what the team wanted to work on, which was *rocket* state estimation and control. Axial drag control surfaces, or "airbrakes", were relatively popular at IREC, due to accurate apogee being a large part of scoring. The team moved to the Launch Canada competition in 2024, and discussions with the organizers made flying active controls, including airbrakes and attitude control, seem feasible. Conceptual designs for an airbrake system on the team's 2024 competition rocket, *Borealis*, were drawn up shortly after the 2023 competition, and by the time the design cycle kicked off in fall it became clear that the scope of airbrakes warranted its own dedicated subsystem.

There was considerable interest in the technical problems Controls was tackling, and the Controls team grew quickly to a similar size as the other rocket subsystems. Airbrakes were successfully integrated into and launched on Borealis in the LC 2025 competition[2]. However, with an apogee of less than 20k feet (due to under-filling of propellant), the rocket undershot the minimum target altitude for actuation of the airbrakes, and thus it was not possible to assess the control performance.

Due to the the difficulties in accurately controlling drag, and more long term team objectives, the decision was made that Controls would not attempt to fly airbrakes again, and instead the team is to develop a method for active roll control. Roll control is another proven technology in the amateur rocketry space. The two main justifications are stability of in-flight footage, and mitigation of roll-pitch coupling effects which are known to pose a significant risk to flight stability at higher altitudes, which the team is pursuing.

[1] The team achieved second place in the 30k SRAD hybrid/liquid class.

[2] The team scored first, and became the first to fly a liquid bipropellant rocket in Canada.

Aurora rocket

The Waterloo Rocketry team is developing the sounding rocket *Aurora* for LC 2025. Aurora is about 4.34m long with a diameter of 20.3cm, has a dry mass of about 51kg and carries about 35kg of propellants. The liquid bipropellant engine is pressure-fed, and uses nitrous oxide as its oxidizer and ethanol as fuel. The engine will burn for about 11s with a maximum thrust of about 10kN, which propels the rocket to about Mach 2.3 and a 16km apogee. Next to the engine components and a parachute recovery system, Aurora carries scientific payloads – and the experimental control system.

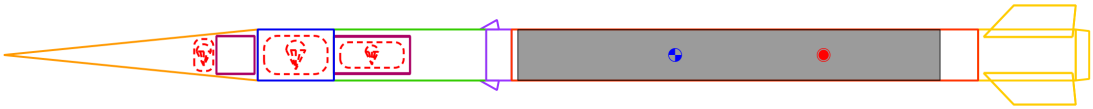


Figure 0.1: Wireframe of the Aurora rocket. From OpenRocket, canards are shown in purple.

Canard roll control

The rocket features an active control system in the form of controlled canards - small, steerable aerodynamic surfaces forward of the tail fins and center of mass. These canards are designed to provide Aurora roll control, the ability to stabilize the rocket around its longitudinal axis and to follow a roll program. A low (or zero) roll rate inhibits *coning*, a phenomena of coupled roll-pitch oscillations which drive an increasing precession of the rocket. Excessive coning may end in the destruction of the rocket in flight. Additionally, a controlled roll motion results in smoother pictures of the onboard cameras.

The successful development of a roll control system using canards may pave the way for a full attitude control system in the future, which should be able to reduce dispersion and increase performance of the next rocket.

Development overview

The canards system can be roughly divided into four projects: actuation mechanics, the processor board and software, the motor control board, and the controller development/simulation – this documentation deals with the latter.

The processor board is designed to interface with the onboard sensors and implement the control algorithms once they are designed. The processor board hardware is largely unchanged from when it was originally developed for airbrakes, and the design of the canards version was completed through the end of December. The motor control board also had heritage to build on from airbrakes, and hardware for it was complete by the end of December.

Extensive firmware design for processor occurred in parallel with the hardware revision. The control design was done in Matlab, for embedded implementation this code is translated to C. The decision was made to translate the code manually (instead of Matlab Coder), for the learning of the team members and to ease later troubleshooting. System validation is performed by a hardware-in-the-loop test, where the implementation of the

control algorithms run live on the actual hardware while interacting with the simulation, and their outputs are checked against the full simulated loop.

The team has existing infrastructure for simulation, with the Flight Dynamics team relying primarily on OpenRocket, a proven rocket simulation software. Flight dynamics also serves as the primary systems engineering team, combining propulsion data to define the geometry and performance targets of the rocket. The simulation was developed in parallel with flight dynamics' own design phases. Canard aerodynamics were developed jointly, with the Flight Dynamics team leading.

The simulation model and the actual controller design were researched and developed largely independently. This ensures that the simulation serves as a valid testing platform for the control algorithms.

Simulation

The Simulink plant model used for development is based on an existing canards project by Project Jupiter (University of São Paulo, Brazil). The model was largely rewritten in regards to aerodynamics, and new components were added to model sensor behaviour and to reflect actuator dynamics more accurately. In order to make team-wide integration easier, the simulation uses as much of the team's flight dynamics data as possible.

The model uses look-up tables for (varying) parameters like drag, thrust, and mass distribution, which are taken directly from the OpenRocket data for maximum parity. The model dynamics implemented in Simulink were validated by comparing the simulated trajectories of both Simulink and OpenRocket.

Controller design

The control development was done using model-based design, as it is a proven and reliable method. As the development was supposed to be as independent from simulation as possible, an effort was made to research and develop flight models from the ground up. Rough rocket performance targets were available in October, from whose it was clear that control loop had to operate in a drastically varying environment (mainly very high velocity variations).

The decision to use LQR synthesis with a gain schedule for the controller, combined with an EKF for the state estimation was made early on. LQR allowed a largely automatic controller design for the thousands of design points of the gain schedule. The EKF was chosen to encompass the non-linear dynamics of rocket flight, and because it is a known approach in aerospace vehicle state estimation. Both of these methods are completely dependent on the fidelity of their model, thus most of the control design effort has been developing these models.

Luckily, resources on aerospace vehicle dynamics and their control are readily available, in form of textbooks, an abundance of papers, and in lectures taken by the author. Most of the model development was done by December, and focus shifted more on the controller and estimator algorithms. Common development, and simulation was largely done in January, and tuning continued while embedded implementation started in February.

Chapter I

Controller and State Estimator Design

Finn Breuer

In this chapter, the rocket flight is modelled, and using this model a control loop consisting of a state estimation algorithm and a state feedback controller is designed. The state estimation is realized with an EKF, and a gain scheduled LQR is used as feedback control.

1 Modelling

In this section the "rocket-flight" system will be modelled. "Modelling" means finding the underlying equations which describe the behaviour of the rocket in flight, its sensors and actuators, the environment, etc., and making assumptions and simplifications to ease the design of the control loop. As the famous (and often repeated by my professors) quote goes

> "All models are wrong, but some are useful" - George Box

Dynamical models of the rocket flight are needed for both the controller synthesis and the estimator algorithm. The rocket model is made up of a system of algebraic and differential equations [3, 4, 5], represented in *state-space* as [4, 5]

$$\dot{x} = f(t, x, u) \tag{1.1}$$
$$y = h(t, x, u) \tag{1.2}$$

with the state x, measurement output y, and control input u. The state (or dynamics) equation f and the output equation h can be nonlinear functions, and may also depend on the time t[1]. The equations are vector-valued, and the variables x, u, y are vectors.

Linear time-invariant (LTI) state-space models can be derived by linearizing the equations f and h around some point (t_0, x_0, u_0), which results in the equations

$$\dot{x} = A(x - x_0) + B(u - u_0) \tag{1.3}$$
$$y = C(x - x_0) + D(u - u_0) \tag{1.4}$$

where the matrices A, C are the Jacobians of f, h with respect to the state x, and similar C, D are Jacobians with respect to the input u.

The state x is a collection of variables which describe the dynamics of the system in a collection of first order differential equations.[2] These states do not have to have a physical

[1] The equations will turn out to not depend explicitly on time, i.e. the differential equations 1.1 may be treated as autonomous.

[2] For example: a mass-spring-damper system has two state variables, position and velocity. These describe the energy stores of the system, potential of the deformed spring and kinetic of the moving mass.

meaning [3], but will in this case as they describe the system well. For aerospace vehicles, often chosen states are the attitude (Euler angles or quaternion), attitude rates, velocity, and position. The input signal u describes how external sources influence the dynamics (and the measurement). This describes behaviour which is not captured by the state alone, for example actuators acting on the system. The output signal y describes how a certain state influences the environment. This mostly encompasses measurements taken from the system by sensors.

1.1 Kinematics and Quaternions

Coordinate frames

The inertial frame f^G is assumed to be the earth-flat geographic frame, with an up-crossrange-downrange orientation (this is more useful than up-north-west). The origin is located below the launch rail, with zero altitude at sea level.

The body-fixed frame f^B is oriented along the main inertia axes of the rocket body, namely forward-starboard-rail (x^+ is forward, and z^+ points toward the launch rail, downrange at launch), as shown in Figure 1.1. The origin of the body-fixed frame is (as typical) located at the body center of gravity.

The sensor frames f^S are aboard the rocket, but rotated and translated against the body frame.

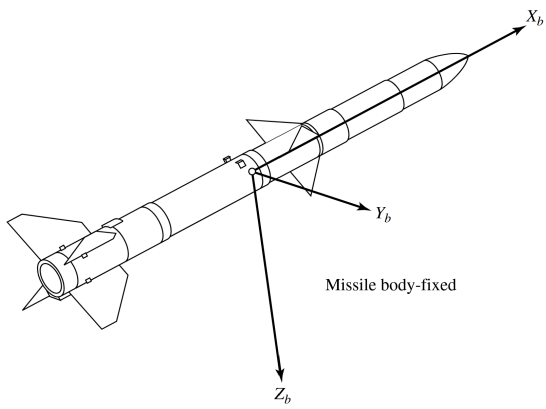


Figure 1.1: Diagram of the body-fixed frame f^B, taken from [6]. (Here a different convention is used to denote the axes, X_b rather than x^B).

The rotational transformation of a vector r between the frames is given by

$$r^G = Sr^B \qquad r^B = S^T r^G \tag{1.5}$$

$$r^B = S_i r^S \qquad r^S = S_i^T r^B \tag{1.6}$$

with the rotation matrix S (see Equation 1.14), and the rotation matrix S_i for the i-th sensor frame. The frames f^G and f^B are shown in Figure 1.2 with their rotation against each other. To simplify the following notations, the velocities, the angular rates, and the external forces and torques are given in the body-fixed frame, and the index ^{B} is omitted.

[3] One could describe a system with its input-output relationship, without assigning a meaning to the variables of the differential equations.

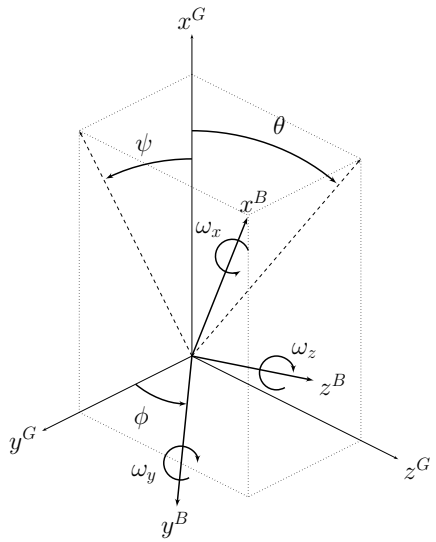


Figure 1.2: Diagram of the attitude coordinate frames with Euler angles and body rates.

Attitude with Quaternions

In aeronautics Euler angles are often used for control design. However, the kinematics of the Euler angles show singularities at certain orientations, and the notation with large rotation matrices from sines and cosines is cumbersome. For this reason the attitude of the rocket will be described by quaternions, as

> "these differential equations are a joy to implement, because they are linear, have no singularities, and number only four" - Peter Zipfel [3]

The Quaternion is defined as

$$q = \begin{bmatrix} q_w & q_x & q_y & q_z \end{bmatrix}^T = \begin{bmatrix} q_w \\ q_v \end{bmatrix} \tag{1.7}$$

which consists of four variables, often named as the real part q_w, and the vector part q_v.[4] The attitude quaternion is normed to $|q| = q^T q = 1$ to describe rotations. This unit constraint has to be enforced in the filter process, e.g. by norming q after integration [7] as $q = q/\text{norm}(q)$, or by Baumgarte stabilization (implemented is the former).

Euler's rotation theorem states that every rotation sequence (or attitude difference) in 3D space can be expressed as a single rotation with angle φ around a single axis n (unit vector) [3]. The quaternion can be understood as that rotation, with

$$q_w = \cos(\frac{\varphi}{2}) \qquad\qquad q_v = n\sin(\frac{\varphi}{2}) \tag{1.8}$$

The inverse of a unit quaternion is simply the conjugate [5, 7], so

$$q^{-1} = \begin{bmatrix} q_w \\ q_v \end{bmatrix}^{-1} = \begin{bmatrix} q_w \\ -q_v \end{bmatrix} \tag{1.9}$$

[4] Formally, the quaternion is a four-dimensional complex number, instead of $a+\mathrm{i}b$ it is $q_w+\mathrm{i}q_x+\mathrm{j}q_y+\mathrm{k}q_z$, where all variable are orthogonal and $\mathrm{i}^2 = \mathrm{j}^2 = \mathrm{k}^2 = \mathrm{ijk} = -1$. This will not really be relevant for implementing the mechanics, but explains some weirdness of the calculations.

To rotate one attitude to another attitude, the attitude quaternions are multiplied using the Hamiltonian product $*$. This can also be applied to rotate vectors, but the rotation matrix S (Equation 1.14) is computationally more efficient. Two quaternions q, r are multiplied to quaternion p as [5]

$$p = q * r = \tilde{Q} \cdot r \qquad \tilde{Q} = \begin{bmatrix} q_w & -q_x & -q_y & -q_z \\ q_x & q_w & -q_z & q_y \\ q_y & q_z & q_w & -q_x \\ q_z & -q_y & q_x & q_w \end{bmatrix} \tag{1.10}$$

with the matrix $\tilde{Q}$ stemming from the Hamiltonian product [5]. To note: this multiplication is not commutative ($q * r \neq r * q$).

The time derivative of the quaternion attitude, with angular rates ω described in the body-fixed frame, has multiple equal expressions [3, 7, 5]

$$\dot{q} = \frac{1}{2} q * \begin{bmatrix} 0 \\ \omega \end{bmatrix} = \frac{1}{2} \tilde{Q} \cdot \begin{bmatrix} 0 \\ \omega \end{bmatrix} = \frac{1}{2} \tilde{\Omega} \cdot q \qquad \tilde{\Omega} = \begin{bmatrix} 0 & -\omega_x & -\omega_y & -\omega_z \\ \omega_x & 0 & \omega_z & -\omega_y \\ \omega_y & -\omega_z & 0 & \omega_x \\ \omega_z & \omega_y & -\omega_x & 0 \end{bmatrix} \tag{1.11}$$

Time integration leads to the discrete dynamics of a quaternion attitude, described by the rotation with an incremental attitude difference [7, 5]

$$q(t + dt) = q(t) * dq(t, dt) \qquad \Longrightarrow \qquad q_{k+1} = \tilde{Q}_k \cdot dq_k \tag{1.12}$$

where for constant angular velocities ω in this short time frame dt, the incremental attitude quaternion dq is [7]

$$dq(t, dt) = dq_k = \begin{bmatrix} \cos(\frac{d\varphi}{2}) \\ dn \sin(\frac{d\varphi}{2}) \end{bmatrix} \qquad d\varphi = \|\omega(t)\| dt \qquad dn = \frac{\omega(t)}{\|\omega(t)\|} \tag{1.13}$$

This is the approximate (truncated Taylor series) closed-form solution of the differential equation 1.11. Its explicit, first order, and maintains the unit constraint, thus this form will be used for time integration of the quaternion, rather than explicit Euler for the other derivatives (see Section A.1.2).

The rotation matrix is used to transform a vector from one attitude to another, i.e. for the transformation from the body-fixed frame to the inertial frame. It can be computed directly from the quaternion [3, 5]. The rotation matrix $S = S_{G \to B}$ (or often $S = S_G^B$) transforms a vector from the ground frame G to the body-fixed frame B, and is computed as

$$S = 2 q_v q_v^T + (q_w^2 - q_v^T q_v) I_3 - 2 q_w \tilde{q}_v \tag{1.14}$$

The inverse transformation, i.e. from a vector in B to a vector in G, is performed by inverting the transformation matrix. Luckily, the rotation matrix is always orthogonal, so $S^{-1} = S^T$, and thus $S_B^G = S^T$. Additionally, the inverse rotation matrix is the rotation matrix of the inverse quaternion, so $S^T(q) = S(q^{-1})$.

The Euler angles *yaw* ψ, *pitch* θ, and *roll* ϕ can be recovered from the rotation matrix elements (or directly from the quaternions). For the rotation sequence yaw $\to$ pitch $\to$ roll (order $z \to y \to x$, rotation around the respective new body axes) the Euler angles are

$$\psi = \text{atan2}(S_{21}, S_{11}) \qquad \theta = \arcsin(-S_{31}) \qquad \phi = \text{atan2}(S_{23}, S_{33}) \tag{1.15}$$

For initialization, the quaternion is set according to Equation 1.8, where φ is the launch angle, and n indicates the launch orientation (pitch or yaw) [5]. A typical initialization could be $\varphi =$ launch angle, and $n = [\,0\ 1\ 0\,]^T$, i.e. the rocket is pitched downrange, and both yaw and roll are defined zero.

Relative motion

Using Newton's second law, the acceleration in the body-fixed frame is [3, 5]

$$\dot{v} = \mathcal{F}/m - \omega \times v + Sg^G \tag{1.16}$$

which accounts for the derivation in the rotating body-fixed frame with the transport term $-\omega \times v$, and the vector of external forces $\mathcal{F}$ (in the body-fixed frame). The gravity vector $g^G = \begin{bmatrix} -g_0 & 0 & 0 \end{bmatrix}^T$ is transformed to the body-fixed frame to account for the gravitational acceleration (magnitude $g_0 \approx 9.81 \frac{\mathrm{m}}{\mathrm{s}^2}$). To note: gravity is thus not a part of the external forces $\mathcal{F}$.

The velocity in the inertial (geographic) frame is [3]

$$\dot{r}^G = S^T v \tag{1.17}$$

With Euler's law, the angular acceleration is [3, 8]

$$\dot{\omega} = J^{-1}(\mathcal{T} - \omega \times J\omega) \tag{1.18}$$

which includes the external torques (or moments) $\mathcal{T}$ (described in the body-fixed frame) and the gyroscopic moment. The inertia tensor J is determined from CAD or experimentally. If the coupling inertia moments are very small, they may be neglected to fill the tensor diagonally as $J = \mathrm{diag}\begin{bmatrix} J_x & J_y & J_z \end{bmatrix}$, where in a rocket cruciform typically $J_x < J_y = J_z$.

1.2 Sensor measurements

All sensors are prone to high frequency noise, low frequency biases, and scaling factors, among other disturbances (such as non-orthogonality of the axes, eigendynamics of the test masses). Noisy measurements are filtered by the EKF process, but the biases also need to be corrected. For the different sensor models, scaling factors B_i are factored in and biases b_i are added.

The Inertial Measurement Units (IMUs) which house all sensors[6], are not mounted at the center of gravity of the rocket body, and the sensor frame is generally not aligned to the body frame. Therefore, a transformation to the body-fixed frame must be formulated to model the vectorized sensor measurements. This includes orientation correction with the rotation matrix S_i, and for the accelerometers distance to the c.g. d_i.
All these corrections are determined during hard calibration (S_i, d_i, B_i are found in workshop testing, where some corrections are hard-coded), and by the initialization filter on the launch pad (b_i, see Section 3.6).

About the notation: the true states of the rocket are mostly denoted as lower case (e.g. ω, $\dot{v}$ p), and thus the corresponding measurements are mostly denoted in upper case (e.g. Ω, A, P). Keep in mind that the measurement and the true value are *not* the same, and noise is not represented in the equations.

[5] Initialization with Euler angles is also possible[3, p. 126], but too much effort to write down here.

[6] Simplified here: an IMU is defined as Gyroscope + Accelerometer, but Magnetometer and Barometer are on the same board/housing.

Angular velocity - Gyroscope

The rate gyroscope measures the angular velocity vector Ω. The measurement is in the rotated sensor frame, and needs to be transformed to the body-fixed frame as[7]

$$\omega = S_\Omega \Omega - b_\Omega \qquad \Omega = S_\Omega^T(\omega + b_\Omega) \tag{1.19}$$

where S_Ω is the rotation vector from body-fixed to the sensor frame. Additionally, the offset biases b_Ω, as determined by the Initilization filter, are corrected for. Note that the the right hand side ($\Omega = ...$) is used in the measurement model.[8] This transformation is done for each IMU separately, with their own $S_{\Omega,i}$ and $b_{\Omega,i}$.

Acceleration - Accelerometer

The accelerometer measures the acceleration vector A. The transformation from the sensor frame to the body frame is computed as [5]

$$a = S_A A - \dot{\omega} \times d_A - \omega \times (\omega \times d_A) \tag{1.20}$$

resulting in the corrected specific force a in the body-fixed frame, with the vector from rocket center of mass to the sensor d_A, and the rotation matrix from the sensor frame to the body frame S_A. This corrects for angular and centrifugal accelerations of a rotating rocket, which show up in points away from the center of gravity but are not "real", i.e. do not change the velocity of the rocket. This transformation is performed for each IMU separately. The resulting velocity derivative in the body-fixed frame is then [3, 5]

$$\dot{v} = a - \omega \times v + Sg^G \tag{1.21}$$

which is the same as Equation 1.16, only the measured specific force a has replaced the calculated forces $\mathcal{F}/m$. The problem of whether to use a modelled external force $\mathcal{F}$ or the measured specific force a is addressed in Section 1.6.

Magnetic field - Magnetometer

The magnetometer measures the local magnetic field vector M. This is mostly determined by Earths magnetic field M_E^G (which is assumed to be constant over the flight, and predetermined by initialization on the ground), but can include disturbances from local sources on the rocket and ground support equipment.

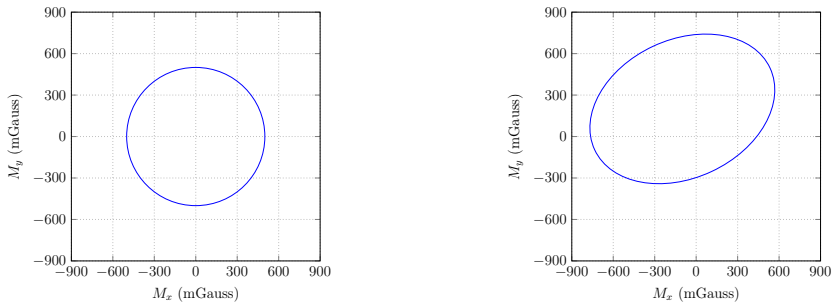
Hard iron disturbances emanate from permanent and electromagnets, which cause an offset bias by vector b_{HI} in the measured field strength as shown in Figure 1.3b.
Soft iron disturbances stem from nearby ferromagnetic materials (predominantly Iron, Nickel, Cobalt), which react to the present magnetic field. This causes stretching of the measured field (shown in Figure 1.3b) by the matrix B_{SI}, which is unity if no soft iron disturbances are present.

The model of the measured magnetic field with hard and soft iron biases b_{HI} and B_{SI} is

$$M = S_M^T(B_{SI} S M_E^G + b_{HI}) \tag{1.22}$$

[7] In some literature, Ω refers to the skew-symmetric matrix of ω. Here, this matrix is expressed as $\tilde{\omega}$ or $(\omega\times)$, while Ω refers to sensor measurements.

[8] One could use the gyroscope measurement as input for the dynamics model, as is done for the accelerometer, in which case the left hand equation would be used.



(a) Disturbance free (b) Hard and soft iron disturbances

Figure 1.3: 2D Visualization of the measured magnetic field [9].

where Earths magnetic field vector M_E^G is rotated into the body-fixed and then the sensor frame with S_M^T. Hard calibration (Section 3.7) must account for both biases and sensor frame rotation. The inverse transformation needed for initialization is

$$M_E^G = S^T(B_{SI}^{-1}(S_M M - b_{HI}) \tag{1.23}$$

Pressure - Barometer

The barometer measures the local air pressure with the sensor bias as

$$P = p + b_P \tag{1.24}$$

Air temperature T cannot be measured reliably from barometric sensor, and will therefore not be used. The atmosphere is modelled in Section 1.3.

Canard angle - shaft encoder

There is a rotary potentiometer mounted on the canard shaft. This encoder measures the canard angle directly, and the resulting sensor signal δ_e is

$$\delta_e = \delta \tag{1.25}$$

Any biases and scaling is calibrated for by the encoder handler, and the resulting signal is just noisy.

Attitude and Heading reference system

Commercial motion sensor units often feature not only an IMU, which provides raw sensor data of accelerometer, gyroscope, and magnetometer, but are also an self-contained *Attitude and Heading Reference System* (AHRS). These AHRS use their own sensor fusion algorithms to provide the 3D sensor attitude, often as Euler angles Θ or Quaternions Q. The sensor model is complex as it contains the sensor models of the internal sensors, and the (often proprietary) filter computation. However, the output can be approximated as the true attitude with additional white noise, so

$$Q = q_Q * q \qquad\qquad q = q_Q^{-1} * Q \tag{1.26}$$

where the sensor attitude is rotated against the rocket frame by the constant q_Q. Commercial AHRS are reliable attitude references, however their accuracy in these highly dynamic flight conditions (such as $\geq$ 10g launch acceleration) remains to be tested.

Satellite navigation

Position in 3D as well as ground speed and heading could be determined by GPS. However, the nominal trajectory goes well outside operational limits of commercially available GPS receivers (a < 4.5g, alt < 18km, v < 515m/s, by the time the rocket is below the acceleration limit it is above the speed limit)[9]. While a GPS may be useful for recovery purposes, it cannot be used during most of the ascent, and therefore the filter will not include GPS.

1.3 Atmosphere and air data

The static pressure p, temperature T, density ρ, and the local speed of sound c of the surrounding air vary with altitude, as shown in Figure 1.4. This air data model is used to predict the barometer measurements and the aerodynamic effects accurately. The air at altitude l is modelled according to the US standard atmosphere [10, 11] as

$$\ell = \frac{R_0 l}{R_0 - l} \tag{1.27}$$

$$T = T_B - k_B(\ell - \ell_B) \tag{1.28}$$

$$p = p_B \exp\left(\frac{-g_0}{RT_B}(\ell - \ell_B)\right) \qquad p = p_B\left(1 - \frac{k_B}{T_B}(\ell - \ell_B)\right)^{\frac{g_0}{Rk_B}} \tag{1.29}$$

$$\rho = \frac{p}{RT} \tag{1.30}$$

$$c = \sqrt{\gamma RT} \tag{1.31}$$

where the index B indicates a base value from Table I.1. The altitude above sea level (geometric altitude) l is converted to the geopotential altitude ℓ for the atmospheric calculations, using the mean Earth radius $R_0 = 6356.77$km. The gravitational acceleration is $g = 9.81\frac{\mathrm{m}}{\mathrm{s}^2}$. The adiabatic index of air is $\gamma = 1.4$, and the specific gas constant of air is $R = 287.0579\frac{\mathrm{J}}{\mathrm{kgK}}$. To calculate the static air pressure, Equation 1.29 is split in two, where the left equation is only used for a zero lapse rate $k_B = 0$.

Layer	Altitude ℓ_B [m]	Pressure p_B [Pa]	Temperature T_B [K]	Lapse rate k_B [$\frac{\mathrm{K}}{\mathrm{m}}$]
Troposphere	0	101325	288.15	0.0065
Tropopause	11000	22632.1	216.65	0
Stratosphere	20000	5474.9	216.65	-0.001
Stratopause	32000	868.02	228.65	-0.0028

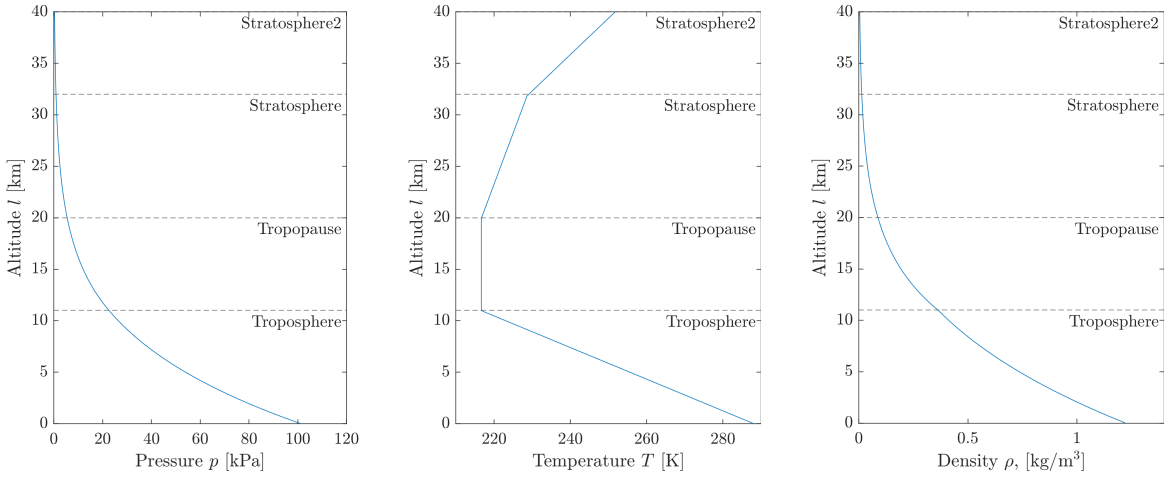
Table I.1: US Standard Atmosphere base data, [10]

Dynamic pressure & Mach number

The airspeed of the rocket is defined as $V_A = |v|$, which assumes no or only little wind disturbances[10]. The dynamic pressure $\bar{p}$ and the Mach number Ma are defined as

$$\bar{p} = \frac{1}{2}\rho V_A^2 \qquad \mathrm{Ma} = V_A/c \tag{1.32}$$

[9] The altitude and velocity limits are imposed by the US International Traffic in Arms Regulations.
[10] These are difficult to predict, and will thus act as disturbances on the rocket-estimator loop.



(a) Pressure (b) Temperature (c) Density

Figure 1.4: Air value variations with altitude, according to the US standard atmosphere model [11].

Where the dynamic pressure is the predominant factor in the aerodynamic forces and moments. As the local speed of sounds varies with altitude, the Mach number for the same airspeed varies at different heights. The Mach number has significant influence on the aerodynamic coefficients, and the location of the center of pressure.

Angle of attack & sideslip

The incidence of the rocket relative to the oncoming airstream is defined with the *angle of attack* α and the *angle of sideslip* β. As any wind gusts act as disturbances, both α and β are defined from the velocity vector of the rocket in the body-fixed frame, as seen in Figure 1.5. The angles are therefore

$$\begin{aligned} &v_x > 0 && v_x = 0 && v_x < 0 \\ &\alpha = \arctan(v_z/v_x) && \alpha = \pi/2 && \alpha = \pi - \arctan(v_z/v_x) && (1.33) \\ &\beta = -\arctan(v_y/v_x) && \beta = -\pi/2 && \beta = -\pi + \arctan(v_y/v_x) && (1.34) \end{aligned}$$

Because the rocket is passively stable to movement in the forward direction (as determined by the Flight Dynamics subsystem), this can be approximated as

$$\begin{aligned} &v_x > 0 && v_x \leq 0 \\ &\alpha \approx v_z/v_x && \alpha \approx \pi/2 && (1.35) \\ &\beta \approx -v_y/v_x && \beta \approx -\pi/2 && (1.36) \end{aligned}$$

1.4 Aerodynamics

The rocket dynamics have external forces and torques acting on the body, which are described here. Note that the forces are described here, but the filter uses accelerometer data in the dynamics model, making the force computations unnecessary. The model could also be used not as a filter, but as a flight simulation, where explicit force equations would be used. The external torques are still important however, as they couple the some unmeasured states to the angular velocity measurements.

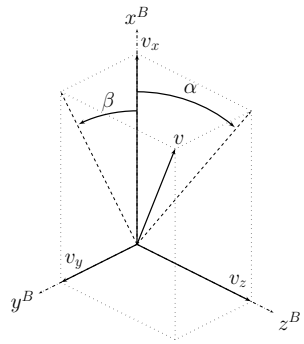


Figure 1.5: Diagram of the aerodynamic angles α and β in the body-fixed coordinate frame

Forces and moments

Most of the external forces and torques acting on the rocket will stem from aerodynamics. The effect of rocket body geometry, including fins and nosecone, can be approximated using Barrowman's method [12].

Any aerodynamic force for each component ^{i} is defined as

$$\mathcal{F}^i = \bar{p}\, C^i_{\dots} A^i \tag{1.37}$$

where the aerodynamic coefficient (lift C_L, normal C_N, drag C_D, roll C_l) and reference area A^i depend on the geometry of the component, and the lift coefficient encompasses information about the incidence angles.

The torque produced by the aerodynamic forces is determined by the vector of the aerodynamic forces, acting through the the *center of pressure* and producing a moment around the center of gravity as

$$\mathcal{T}^i = \mathcal{F}^i \times d^i \tag{1.38}$$

where d^i is the vector from the center of pressure to the center of gravity (direction matters!).

The total aerodynamic forces and moments are

$$\mathcal{F}_A = \sum_i \mathcal{F}^i \qquad\qquad \mathcal{T}_A = \sum_i \mathcal{T}^i \tag{1.39}$$

For reference, later used coefficients are listed in Table I.2. *Derivatives* are often used in aerospace control literature and encompass the force/torque and the corresponding inertia, partially differentiated around a variable $_i$.

Symbol	Name
$C^{\text{can}}_{L_\delta}$ (in state notation C_L)	Lift coefficient, canards
L_δ	Roll control derivative
$C^{\text{fins}}_{l_{\omega_x}}$	Roll damping coefficient, fins
L_{ω_x}	Roll damping derivative, fins
C_{N_α}	Normal force coefficient, entire rocket

Table I.2: Aerodynamic parameters

Canard airfoil theory

Most of the in-depth development has been done by Luca Scavone, lead of the flight dynamics subsystem [13]. The coefficient of lift is approximated using thin airfoil theory [11]. The canards have a triangular (*delta* Δ) shape, with a high *sweep angle* Λ. The lift coefficient depends on the Mach number of the rocket, as the flow behaviour starts to change near sonic speed. Protruding geometries of the rocket body, such as the canards, have shock waves attached to them if the surrounding flow is supersonic. At higher speeds, these shock waves angle more and more backwards. The characteristic Mach angle is defined as $\mu = 0$ for $\mathrm{Ma} < 1$ and for $\mathrm{Ma} \geq 1$ as [11]

$$\mu = \arccos(\frac{1}{\mathrm{Ma}}) \tag{1.40}$$

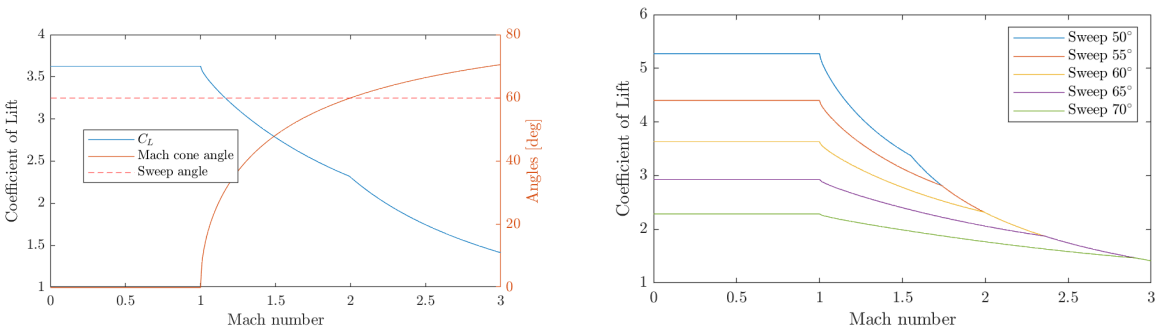
For a subsonic leading edge $\mu \leq \Lambda$ (i.e. the canard is more acute than the Mach cone), the coefficient of lift is empirically determined [11] as Equation 1.42. When the Mach cone is steeper than the canard leading edge $\mu > \Lambda$, the coefficient of lift follows a simpler law [11] in Equation 1.43.

$$\mathrm{Ma} < 1: \qquad C_{L_\delta} = 2\pi \cot(\Lambda) \tag{1.41}$$

$$\mu \leq \Lambda: \qquad C_{L_\delta} = \frac{2\pi^2 \cot(\Lambda)}{\pi + \eta(0.38 + 2.26\eta - 0.86\eta^2)} \quad \text{with} \quad \eta = \frac{\cot(\Lambda)}{\cot(\mu)} \tag{1.42}$$

$$\mu > \Lambda: \qquad C_{L_\delta} = \frac{4}{\sqrt{\mathrm{Ma}^2 - 1}} \tag{1.43}$$

For small deflection $\delta < 10°$ of the canard surfaces, the coefficient of lift is proportional to the deflection angle, so $C_L = C_{L_\delta}\delta$. The equations for the coefficient of lift are shown in the Figures 1.6. Figure 1.6a shows the coefficient C_{L_δ} and Mach angle μ for an exemplary sweep angle of 60°,



(a) Lift coefficient and Mach cone angle vs. Mach number, for 60° sweep angle

(b) Lift coefficients vs. Mach number, for different sweep angles

Figure 1.6: Coefficient of lift for swept delta airfoils

Roll forces and roll moments

The resulting force of one canard is $\mathcal{F}_1^{\mathrm{can}} = \bar{p}\, C_{L_\delta}^{\mathrm{can}} \delta A_1^{\mathrm{can}}$, which is orthogonal to the x^B vector, and acts on the center of pressure of the canard surface.

As the canards are of equal size, and move in equal and opposite angles, the normal force on the rocket body cancels out (for small angles of attack of the rocket body), it remains

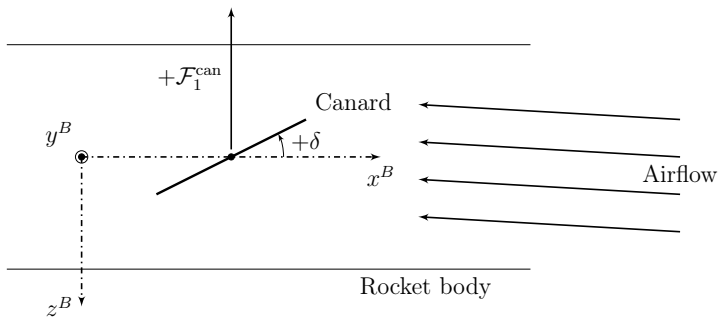


Figure 1.7: Diagram of the canard angle δ in the body-fixed coordinate frame. Modelled after [6].

a net moment about the rocket x-axis. The forces and moments produced by the canards are thus

$$\mathcal{F}^{\text{can}} = \begin{bmatrix} 0 \\ 0 \\ 0 \end{bmatrix} \qquad \mathcal{T}^{\text{can}} = \begin{bmatrix} \bar{p}\, C_{L_\delta}^{\text{can}} \delta A^{\text{can}} d_y^{\text{can}} \\ 0 \\ 0 \end{bmatrix} \tag{1.44}$$

where A^{can} is the total planform area of all canards and d_y^{can} is the distance between the rocket roll axis x^B and the center of pressure of one of the canards (i.e. the moment arm). The instantaneous roll acceleration of the canards can be written with the roll control derivative L_δ as

$$\dot{\omega}_x = \mathcal{T}_x^{\text{can}} / J_x = L_\delta\, \delta \qquad L_\delta = \bar{p}\, C_{L_\delta}^{\text{can}} A^{\text{can}} d_y^{\text{can}} / J_x \tag{1.45}$$

which simplifies notation of the roll dynamics as shown in Section 1.7.

The roll forces of the rocket body are dominated by fin effects, namely roll forcing from misalignment, and roll damping from the fin wake of a rolling rocket. The roll forcing is difficult to predict if the fin misalignment is not exactly know, so any forcing will be treated as disturbances acting on the rocket. The roll forcing is therefore neglected, but the roll damping is

$$\mathcal{F}^{\text{fins}} = \begin{bmatrix} 0 \\ 0 \\ 0 \end{bmatrix} \qquad \mathcal{T}^{\text{fins}} = \begin{bmatrix} \bar{p}\, C_{l_{\omega_x}}^{\text{fin}} \omega_x A^{\text{fins}} d_y^{\text{fins}} \\ 0 \\ 0 \end{bmatrix} \tag{1.46}$$

The instantaneous roll acceleration from roll damping of the fins can be written with the roll damping derivative L_{ω_x} as

$$\dot{\omega}_x = \mathcal{T}_x^{\text{fins}} / J_x = L_{\omega_x}\, \omega_x \qquad L_{\omega_x} = \bar{p}\, C_{L_{\omega_x}}^{\text{fins}} A^{\text{fins}} d_y^{\text{fins}} / J_x \tag{1.47}$$

Normal forces and moments

Barrowman's method can be employed to derive the location of the center of pressure and the normal force coefficient, for each component of the rocket body [12, 14]. Accurate parameters are determined by the Flight Dynamics subsystem.

Important effects are aerodynamic drag, the total normal force and moment, the roll control and roll damping coefficients, and their respective centers of pressure.

The normal forces and the resulting pitch/yaw torques are

$$\mathcal{F}^{\text{normal}} = \begin{bmatrix} 0 \\ \bar{p}\, C_{N_\alpha} \beta A^{\text{ref}} \\ \bar{p}\, C_{N_\beta} \alpha A^{\text{ref}} \end{bmatrix} \qquad \mathcal{T}^{\text{normal}} = \begin{bmatrix} 0 \\ \bar{p}\, C_{N_\alpha} \alpha A^{\text{ref}} d^{\text{ref}} \\ \bar{p}\, C_{N_\beta} \beta A^{\text{ref}} d^{\text{ref}} \end{bmatrix} \tag{1.48}$$

1.5 Actuators

The mechanical design of the canard actuating system has been done by Ben Pickens, Co-lead of the controls subsystem [15]. The actuation system, shown in Figure 1.8, rotates two canards in opposite directions by equal amounts, thus providing a roll-only torque on the rocket. The actuation is provided by a servo motor, driving the canard shafts via bevel gears. It acts as a position control element, which provides a calculated torque to reach the commanded shaft angle. To provide accurate position feedback to the controller / state estimation, rotary potentiometers are mounted on the canard shafts.

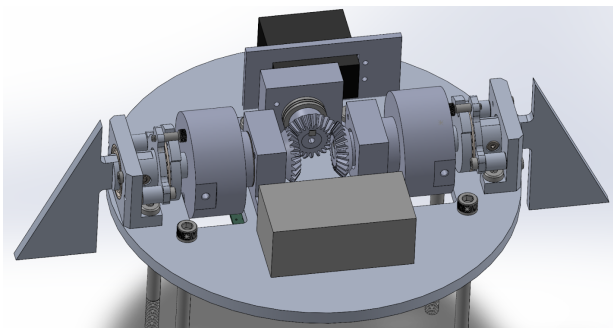


Figure 1.8: Mechanical design of the canards actuation assembly [15].

Actuator dynamics

The internal control loop of the servo has some higher order dynamics, non-linearity in a maximum angle and limited speeds, and some dead-time before it starts responding. A reasonable approximation to use for the controller and state estimation models would be a first or second order linear system, the former is chosen to keep the state vector small (and thus the filter computations shorter).

The first order system with the actual canard angle δ, the commanded servo angle δ_u, and the gear ratio between the servo and the canard β_δ, has the form

$$\dot{\delta} = -\alpha_\delta(\delta - \beta_\delta \delta_u) \tag{1.49}$$

The dynamics are dominated by the time constant α_δ, which describes how fast the canard angle responds to a servo command. Solving the ODE results in the step response

$$\delta(t) = (1 - \mathrm{e}^{-\alpha_\delta t})\beta_\delta \delta_u \tag{1.50}$$

Notably, when no non-linearity is present, i.e. as long as the servo speed limit is not reached, the response time does not depend on the commanded angle δ_u.

The first order response is fitted to the measured actuator response by adjusting the time constant. The gear ration is assumed to be $\beta_\delta = 1$ from now on, later equations need to be adjusted if that should change.

Todo. Insert plots here.

1.6 Estimator model equations

The rocket model is made up of algebraic and differential equations [3, 4, 5], represented in *state-space* as [4, 5] (Equations 1.1 and 1.2)

$$\dot{x} = f(t, x, u)$$
$$y = h(t, x, u)$$

with the state x, measurement output y, and control input u. Here the state equation f and the output equation h are nonlinear functions.

States x

From textbooks [3, 5, 16], lectures [17], papers [18, 19], the industry standard for choosing states of aeronautical vehicles is using attitude angles, attitude rates, velocity, and position.

Here, position is reduced to altitude only, as longitude and latitude are not required and cannot be measured without GPS[11]. The entire state vector is then

$$x = \begin{bmatrix} q & \omega & v & l & C_L & \delta \end{bmatrix}^T \tag{1.51}$$

with the attitude represented as quaternions $q = \begin{bmatrix} q_w & q_x & q_y & q_z \end{bmatrix}^T$, the angular velocity in the body-fixed frame $\omega = \begin{bmatrix} \omega_x & \omega_y & \omega_z \end{bmatrix}^T$ (body rates)[12], the velocity in the body-fixed frame $v = \begin{bmatrix} v_x & v_y & v_z \end{bmatrix}^T$, and the altitude over sea level l. Additionally, the canards have the coefficient of lift C_L as we are unsure about the aerodynamic behaviour of the canards (and as a state, the coefficient can be estimated), and the angle-of-attack δ which is used to carry first order actuator dynamics.

Measurements y

The measurements arrive in the measurement vectors

$$y_{IMU} = \begin{bmatrix} \Omega & M & P \end{bmatrix}^T \tag{1.52}$$
$$y_{encoder} = \begin{bmatrix} \delta_e \end{bmatrix} \tag{1.53}$$
$$y_{AHRS} = \begin{bmatrix} Q \end{bmatrix} \tag{1.54}$$

with measured body rates from the rate gyroscope $\Omega = \begin{bmatrix} \Omega_x & \Omega_y & \Omega_z \end{bmatrix}^T$, the magnetic field measurement $M = \begin{bmatrix} M_x & M_y & M_z \end{bmatrix}^T$ (which can be transformed to attitude[13]) from the magnetometer, additionally the static air pressure P is provided by the barometer (the temperature T is not used, as its measurement is unreliable). The accelerometer data A will be provided to the filter in the input vector, as this simplifies calculations.[14] The encoder signal δ_e also arrives in its measurement vector, and so does a possible attitude reference from an AHRS. Each one of these measurement vectors has its own measurement model.

[11] Commercial GPS lockouts are expected at our speed and altitude, to prevent guided missile design.

[12] In literature the body rates are often depicted as $\omega_x = p$, $\omega_y = q$, and $\omega_z = r$. As this may be confusing, we'll stick with ω_i for the rates in this documentation.

[13] The inverse transformation is not unique as it is invariant about the field direction, but we can't use the accelerometer in free flight to find the gravity vector.

[14] This is quite common practice

Inputs u

The control input vector is the commanded canard angle to the actuator δ_u and, as stated in the previous paragraph, the accelerometer data $A = \begin{bmatrix} A_x & A_y & A_z \end{bmatrix}^T$.

As the acceleration is not a state, it is used in the state prediction step instead of the correction step. The measured acceleration A enters the prediction function as a control input. As these measurements do not enter the filter in the classical way, their weighting of the expected covariances has to happen in the prediction model. The transformation to the body-fixed body-centered frame from Equation 1.20 is performed first, after which the a_i will be weighted and then fed to Equation 1.21.

$$u = \begin{bmatrix} \delta_u & A \end{bmatrix}^T \tag{1.55}$$

The actual canard angle is represented as the state δ, to account for actuator dynamics. The computation from desired canard angle to servo command should be performed in the controller.

Dynamics model

$$\dot{\hat{x}} = f(t, \hat{x}, u)$$

predicts the time derivative of the state vector, $\dot{\hat{x}}$ (Equation 1.51), using the control input u (Equation 1.55), and the following model equations:

- Attitude: 1.11
- Rates: 1.18
- Velocities: 1.21 for filtering and 1.16 for simulation
- Position/Altitude: 1.17
- Canard coefficient: 1st order dynamics $\dot{C}_L = -\alpha_c(C_L - C_{L,\,\text{theory}})$, airfoil theory Equations 1.42 and 1.43
- Canard angle: 1st order dynamics $\dot{\delta} = -\alpha_\delta(\delta - \delta_u)$

Measurement model

$$\hat{y} = h(t, \hat{x}, u)$$

predicts the sensor outputs $\hat{y}$ from the current state estimate $\hat{x}$, and the model equations:

- Gyroscope: 1.19
- Accelerometer: used in state prediction as 1.20 and 1.21
- Magnetometer: 1.22
- Barometer: 1.28 to 1.31

1.7 Controller model equations

The model used for controller synthesis must be linear, time-invariant, and controllable. Linear time-invariant state-space models can be derived by linearizing the equations f and h around some design point (t_0, x_0, u_0), which results in the equations (Equations 1.3 and 1.4)

$$\dot{x} = A(x - x_0) + B(u - u_0)$$
$$y = C(x - x_0) + D(u - u_0)$$

These equations may be employed around a collection of design points $\{t_s, x_s, u_s\}$ do account for varying parameters of the matrices (A, B, C, D).

As the model must be controllable, it makes some simplifications compared to Section 3. This is because the actuating canards can only produce a roll torque, i.e. a moment about the x-axis, but cannot directly influence other states, e.g. pitch and yaw rates or altitude. This results in the full model from Section 1.6 not being controllable, i.e. we cannot use the pole placement method to arbitrarily choose our desired dynamics. Additionally, the full uncontrollable model includes uncontrollable modes, which make analysis of the control loop more cumbersome. To solve this problem, the "full" state dynamics are projected unto the controllable subspace – i.e. the number of states are reduced to those that can be influenced, and the model is simplified.

Simplifications

The cross coupling term $\omega \times J\omega$ is neglected, as the rocket is assumed to have an approximately diagonal J, and the roll dynamics are much faster than the pitch/yaw dynamics. The attitude kinematics of the quaternion are neglected to assume that the rate of the roll angle is the roll rate, so $\dot{\phi} = \omega_x$. This roll angle ϕ is computed from the quaternions in the Estimator; $\phi = \text{atan2}(S_{23}, S_{33})$ as shown in Equation 1.15 (expressed here as entries of the rotation matrix).

The altitude and the velocity vector are not directly controllable, so they are removed from the controller model as well. To preserve the aerodynamics information, the velocity and altitude are combined to the dynamic pressure $\bar{p}$ acting on the rocket. The dynamic pressure and the canard coefficient C_L are not states, but variable parameters of the model matrix A. They are combined to the *flight condition*

$$x_{FC} = \begin{bmatrix} \bar{p} & C_L \end{bmatrix} \tag{1.56}$$

State-space representation

The simplifications leave three state variables for the controller roll model, being roll angle, roll rate, and the canard deflection angle. Thus the state vector x_R of the roll model is

$$x_R = \begin{bmatrix} \phi & \omega_x & \delta \end{bmatrix}^T \tag{1.57}$$

As input to the system serves the commanded canard angle δ_u, and all states are available as outputs due to the state estimation. The resulting state-space model is then

$$\dot{x}_R = A(x_{FC})\, x_R + B\delta_u \qquad\qquad y_R = Cx_R \tag{1.58}$$

with the matrices

$$A = \begin{bmatrix} 0 & 1 & 0 \\ 0 & L_{\omega_x} & L_\delta \\ 0 & 0 & -\alpha \end{bmatrix} \qquad B = \begin{bmatrix} 0 \\ 0 \\ \alpha \end{bmatrix} \qquad C = \begin{bmatrix} I_3 \end{bmatrix} \tag{1.59}$$

where the roll control derivative is $L_\delta(\bar{p}, C_L)$, a function of dynamic pressure and the canard coefficient of lift, i.e. the flight condition. Roll damping is modelled by the roll damping derivative $L_{\omega_x}(\bar{p}, C_{L\omega_x})$. Roll damping originates almost entirely from fixed fins which resist turning - for freely rolling fins (a spin can), the damping derivative should be zero. The 1st order actuator dynamics are accounted for with the δ state, and are characterized by the time constant α, whose value should be determined by experimental system identification. Any gear ratio between servo motor and canards may be expressed as a factor of the bottom entry of B (here set to 1).

2 Controller

The controller is responsible for stabilizing the rocket state, attenuating any disturbances, and driving it to some desired value. In their configuration the canards can only provide a torque around the roll (x) axis of the rocket. This imposes some constraints on the controller design, as the whole state 1.51 is under-actuated. The model used for controller design is thus simplified to the state 1.57, using a linear parameter-varying system.

The controller chosen for the canards is linear state feedback, where the feedback gains K are computed by the *Linear Quadratic Regulator* (LQR) method [20, 21]. The current state is determined by the state estimation algorithm (the Estimator is treated in Section 3), as seen in Figure 2.1. The loop is a two-degree-of-freedom design, where the reference signal is prescaled by a feedforward (reference) gain K_{pre}. To account for the non-linearity of the rocket dynamics the gains are selected from a gain schedule, which is a look-up table of gains for different parameters of the linear model from Equations 1.57, 1.58, and 1.59.

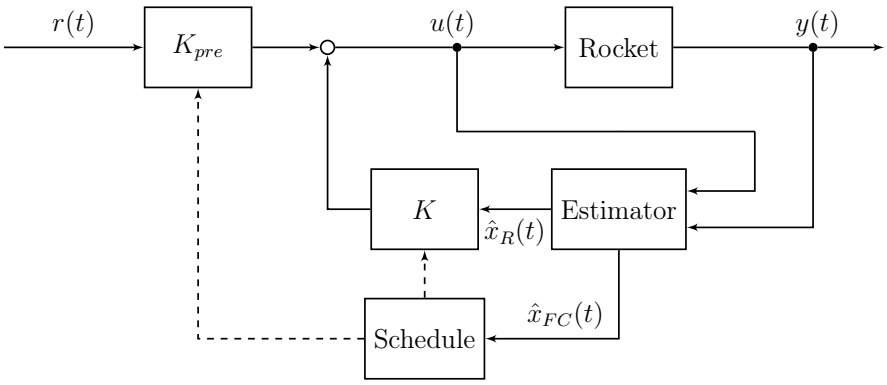


Figure 2.1: Block diagram of the control loop. 2-DOF estimator-based state feedback [20] with gain scheduling.

2.1 Controller roll model

The dynamic pressure and the canard coefficient C_L are the scheduling variables of the controller. The concocted state x_R for the roll model (Equation 1.57) and the flight condition (Equation 1.56) are

$$x_R = \begin{bmatrix}\phi & \omega_x & \delta\end{bmatrix}^T \qquad\qquad x_{FC} = \begin{bmatrix}\bar{p} & C_L\end{bmatrix}$$

The state-space model is (Equations 1.58)

$$\dot{x}_R = A\,x_R + B\delta_u \qquad\qquad y_R = Cx_R$$

with the matrices (Equations 1.59)

$$A(x_{FC}) = \begin{bmatrix}0 & 1 & 0\\ 0 & L_{\omega_x} & L_\delta\\ 0 & 0 & -\alpha\end{bmatrix} \qquad B = \begin{bmatrix}0\\0\\\alpha\end{bmatrix} \qquad C = \begin{bmatrix}I_3\end{bmatrix}$$

where the roll control derivative is $L_\delta(\bar{p},\, C_L)$, and the roll damping derivative is $L_{\omega_x}(\bar{p},\, C_{L\omega_x})$. The matrix C combines the states of the system to an output signal, and can be set arbitrarily for response analysis. For the case $C = \begin{bmatrix}I_3\end{bmatrix}$, the output signal is the entire state vector $y(t) = x(t)$, for the case $C = \begin{bmatrix}1 & 0 & 0\end{bmatrix}$, the ouput signal is $y(t) = x_1(t) = \phi$.

The (approximately in continuous time) closed loop is

$$\dot{x}_R = (A + BK)x_R + BK_{pre}r \qquad\qquad y_R = Cx_R \tag{2.1}$$

Controllability

To determine if pole placement is possible, which is a necessary condition for LQR [20], the roll model is checked for controllability. The controllability matrix is

$$\mathcal{C} = \begin{bmatrix} B & AB & A^2B \end{bmatrix} \tag{2.2}$$

which has full rank for all $L_\delta \neq 0$, i.e. there exists a feasible feedback gain for every flight condition, except for zero velocity ($\|v\| = 0$) or a zero coefficient of lift ($C_L = 0$).

2.2 Control law and tuning

To compute the desired canard angle $u = \delta_u$, the the current rocket roll state as determined by the state estimation is used. The control law is a state feedback, combined with a feedforward part for the reference signal r as

$$u(t) = K\hat{x}_R(t) + K_{pre}r(t) \tag{2.3}$$

where the gains $K = \begin{bmatrix} K_\phi & K_{\omega_X} & K_\delta \end{bmatrix}$ are determined by performing the LQR method around a linear model

$$K(\hat{x}_{FC}) = -\texttt{lqr}(A(\hat{x}_{FC}), B, Q_c, R_c) \tag{2.4}$$

with Matlab's `lqr` function. For the discrete controller, the function $\texttt{lqrd}(A, B, Q, R, dt)$ is used to directly account for discrete dynamics with sampling time dt, but using a continuous-time system (A,B). To note: for fast sampling times (much higher sampling frequency than the bandwidth of the model), the feedback gains of `lqr` and `lqrd` become approximately equal. The method is tuned by adjusting the weighting matrices Q_c and R_c. As shown in section 2.3, the matrix Q_c contains the weights for penalizing the state error, while the matrix R_c weighs the expended control effort.

The scalar reference gain K_{pre} is computed for a steady state gain of 1, as

$$K_{pre}(\hat{x}_{FC}) = \frac{-1}{C\left[A(\hat{x}_{FC}) + BK(\hat{x}_{FC})\right]^{-1}B} \tag{2.5}$$

where C depends on the variable to be tracked. With $C = \begin{bmatrix} 1 & 0 & 0 \end{bmatrix}$, the channel $r \to \phi$ is selected, which results the roll angle tracking the reference. The channel $r \to \omega_x$ could also be selected via $C = \begin{bmatrix} 0 & 1 & 0 \end{bmatrix}$, this would result in tracking of the roll rate.

The gains K and K_{pre} can be precomputed for each combination of parameters $A(\hat{x}_{FC})$. As gain scheduling is used to account for slowly-varying parameters, the gains are be computed for each design point x_s, and combined to a look-up table (see Section 2.4). This way the implemented controller does not need to compute the gains with the LQR method, but can just select the gains from the gain scheduling table.

2.3 Optimal control theory

This section is here for completeness, it is not relevant for implementation. The LQR method uses the weighting matrices Q and R to solve the linear quadratic optimization problem [21] [15]

$$\min V = \int_0^\infty (x^T(t)Qx(t) + u^T(t)Ru(t))dt \tag{2.6}$$

for which the solution is of the form

$$V^* = x^T P x \tag{2.7}$$

In infinite time, P satisfies the algebraic Ricatti equation

$$PA + A^T P - PBR^{-1}B^T P + Q = 0 \tag{2.8}$$

which results in the control law

$$u^*(t) = Kx(t) \tag{2.9}$$

$$K = -R^{-1}B^T P \tag{2.10}$$

The resulting controller for the linear system is optimal in the trade-off of minimizing the state error $x(t)$ and control effort $u(t)$ [20]. The closed-loop is guaranteed to be stable [21], and has margins of $[\infty, 0.5]$ in gain and $60°$ in phase. This guarantee is only valid however for linear systems with full state measurements [22]. The control law is still useful for controlling nonlinear and observed systems, as the robustness properties can be partially preserved [21].

2.4 Gain scheduling

As the models from canard angle to roll rate are nonlinear and highly dependent on some slowly-varying parameters (the *flight conditions*), a gain-scheduling scheme will be implemented. The design points x_s of the schedule are characterized by these slowly-varying flight conditions $x_{FC} = \begin{bmatrix}\bar{p} & C_L\end{bmatrix}$, being the aerodynamic pressure $\bar{p}$, and the lift coefficient of the canards C_L. The simplified model is evaluated around these design points using the current state estimate $A(\hat{x}_{FC})$, to form the matrix A. As the LQR method produces guaranteed robustness margins [22, 20, 21], the interpolation between two close design points should always produce a stable control loop.

A large variety of design points are chosen, both along the nominal flight profile, but also encompassing a larger flight envelope to accommodate off-nominal trajectories. Design points for the condition $L_\delta = 0$ are omitted from the tuning process, as the system is not controllable there. The control gains for all design points are computed beforehand and stored in a look-up table. In flight, the control algorithms interpolate the gains from the look-up table according to the current state estimate. The pre-computed feedback gains for the roll model are shown in Figure 2.2, plotted over the scheduling variables $\bar{p}$ and C_L. To note is the discontinuity at $\bar{p} = 0$ and especially at $C_L = 0$, where no feasible design points exist. The feedback gains are interpolated over this split.

[15] I think this came out better than the estimation theory, please be kind Prof. Werner, Prof. Faulwasser

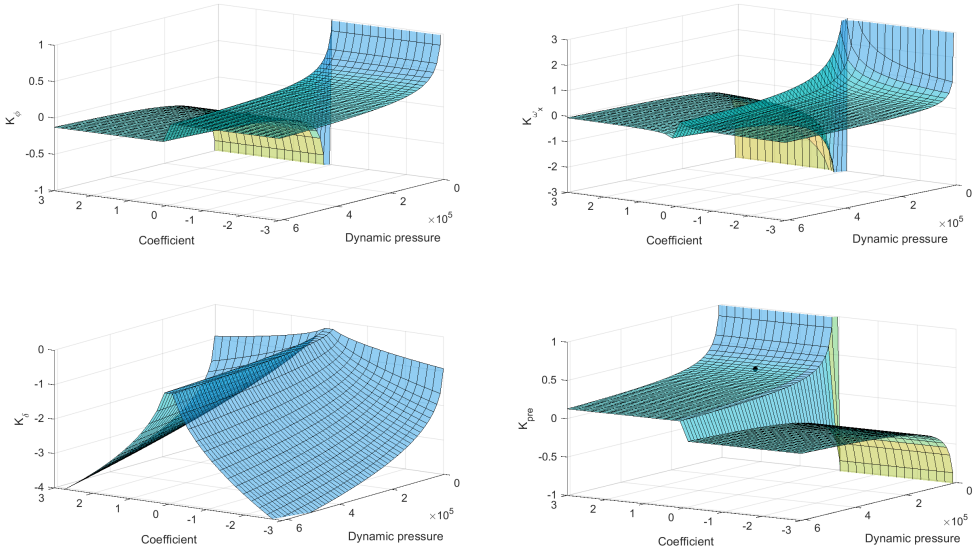


Figure 2.2: Scheduling surfaces along dynamic pressure $\bar{p}$ and canard coefficient of lift C_L. Feedback gains K_ϕ, K_p, K_δ and reference gain K_{pre}.

2.5 Response analysis

To analyze some aspects of the control loop, the closed-loop responses are analyzed. The closed-loop is technically a sampled-data system, with continuous time (the rocket) and discrete time (estimator and controller) dynamics. For a high sampling rate (short time periods between control outputs), the discrete-time and the continuous-time closed-loop are approximately equal. The continuous-time dynamics are (from Equation 2.1) $\dot{x}_R = (A + BK)x_R + BK_{pre}r$ and $y_R = Cx_R$, with $C = \begin{bmatrix} 1 & 0 & 0 \end{bmatrix}$ for the $r \to \phi$ channel.

The weighting matrices are chosen as

$$Q_c = \begin{bmatrix} 10 & & \\ & 1 & \\ & & 10 \end{bmatrix} \qquad R_c = (1 \cdot 10^{-3})\,\bar{p} \tag{2.11}$$

The closed-loop step response for a collection of parameters (here varying velocities) is shown in Figure 2.3.

The Bode plots of the closed-loop for a collection of parameters (here varying velocities) is shown in Figure 2.4.

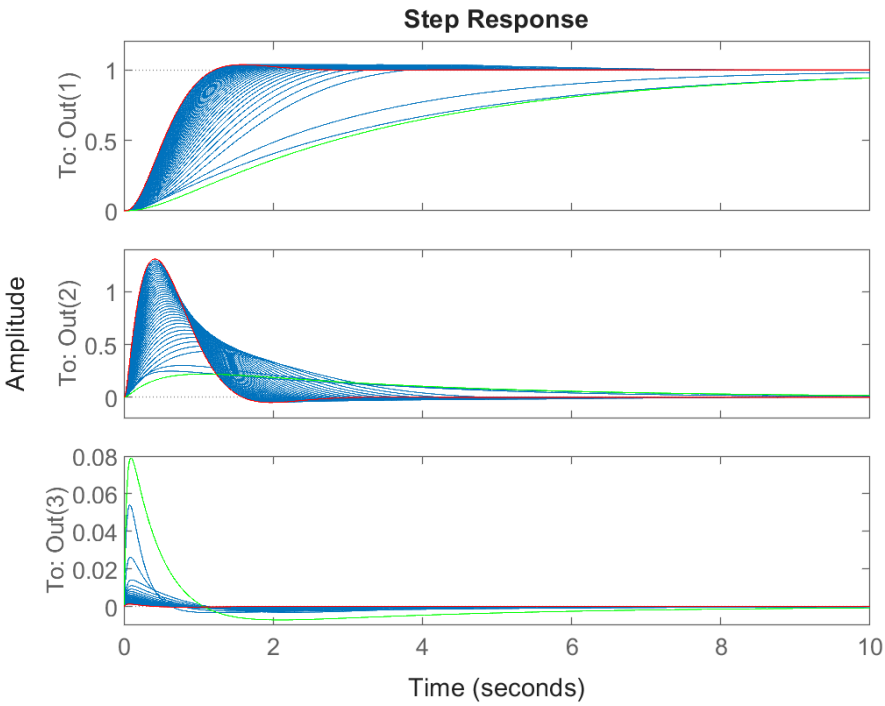


Figure 2.3: Step responses of the closed-loop system for velocities between 30m/s (green) and 800m/s (red), $l = 1000$m, $C_L = 1.5$. Outputs: 1:ϕ, 2:ω_x, 3:δ.

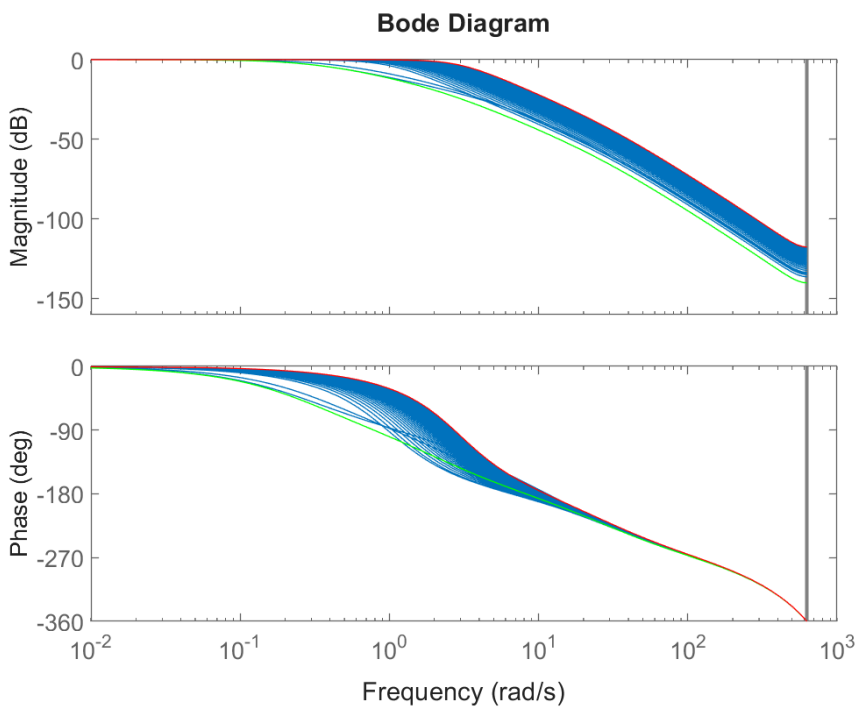


Figure 2.4: Frequency response of the closed-loop system for velocities between 30m/s (green) and 800m/s (red), $l = 1000$m, $C_L = 1.5$. The response is cut off above the sampling frequency of the processor.

3 Estimator

The state estimation algorithm is responsible for accurately determining the current rocket state. The rocket state is a set of data points which describe the condition of the rocket dynamics, and is made up of the rocket attitude, velocity, altitude, aerodynamic coefficients (coefficients are not quite certain due to the lack of available windtunnel testing) and other air data. As the underlying dynamics are nonlinear in some cases, an *Extended Kalman Filter* (EKF) is used to estimate the current state. The EKF algorithm uses these modelled dynamics to

1. predict the current state from the last state and control inputs (desired canard angles) to the rocket, and then
2. correct the state using measurements from onboard sensors, including rate gyroscopes, magnetometers, accelerometers, barometers.

This whole process is a *Bayes filter*, where the state estimate has an associated belief in the accuracy of the estimate. Each iteration of the filter uses the the priori (old) state estimate and belief, to compute/update the posteriori (new) estimate and belief.

3.1 EKF algorithm

The EKF thus propagates a state estimate $\hat{x}$ in discrete time steps, along with the belief in form of the covariance matrix P. While the state is the mean estimate, the covariance matrix represents the "uncertainty" of the state estimate and their correlation, i.e. higher values represent more uncertainty.

For its algorithm the EKF uses the continuous-time prediction model f and the discrete-time measurement model h. However, the EKF is running on a microprocessor in discrete time, so the continuous-time dynamics have to be solved to form a discrete-time update. Together with the solution of the previous filter iteration, this yields the *Initial Value Problem* (IVP)

$$\hat{x}_k^- = \underbrace{\int_{t-dt}^{t} \dot{\hat{x}}(t,x,u)d\tau}_{f_D(x,u)} \qquad \hat{x}(t-dt) = \hat{x}_{k-1}^+ \tag{3.1}$$

The discrete solution $\hat{x}_k^-$ of the IVP is computed by numerical integration (see Section A.1.2), which can be as simple as *explicit Euler*: $x_k = x_{k-1} + T \cdot \dot{x}$. As the IVP is solved in each filter iteration, the integration bounds are the timestamps of the previous and current iteration, with the sampling rate T. The numerical solution (of the integral) is then the discrete update function f_D.

Of great importance are also the the Jacobians of the functions f_D and h, which can be understood as the *sensitivity* of the nonlinear equations f_D and h to variations of the state. Essentially, the Jacobians

$$F_D(x,u) = \frac{\partial f_D}{\partial x} \qquad H(x) = \frac{\partial h}{\partial x} \tag{3.2}$$

indicate how much the outputs of the functions f_D and h change for small changes of each state variable. As the the filter is computed at sufficiently short (see Section 3.5) time intervals, the state does not change greatly between each computation step. The functions

f_D and h can be approximated by linearization, and the covariance is propagated using the resulting Jacobians.

Filter process

One complete iteration of the algorithm of the discrete EKF is [4, 21, 18] [16]

Predict:

$$\hat{x}_k^- = f_D(\hat{x}_{k-1}^+, u_{k-1}) \tag{3.3}$$

$$P_k^- = F_D(\hat{x}_k^-, u_{k-1}) P_{k-1}^+ F_D^T(\hat{x}_k^-, u_{k-1}) + Q_e \tag{3.4}$$

Correct:

$$K_k = P_k^- H^T(\hat{x}_k^-) \left[H(\hat{x}_k^-) P_k^- H^T(\hat{x}_k^-) + R_e \right]^{-1} \tag{3.5}$$

$$\hat{x}_k^+ = \hat{x}_k^- + K_k \left[y_k - h(\hat{x}_k^-) \right] \tag{3.6}$$

$$P_k^+ = \left[I - K_k H(\hat{x}_k^-) \right] P_k^- \left[I - K_k H(\hat{x}_k^-) \right]^T + K_k R_e K_k^T \tag{3.7}$$

where the measurements are used for the state correction, while the control input is used for prediction. Here the prediction part consists of the (now integrated) discrete-time dynamics (Equation 3.3) and the associated covariance in discrete-time (Equation 3.4).

The correction part computes the filter gain (or Kalman gain) K_k using the covariance of the estimate P and the sensitivity of the measurement model H (Equation 3.5). As the measurement model is discrete, the computation of the predicted measurements ($\hat{y} = h(t, \hat{x}_k^-)$) and the correction of the state estimate (Equation 3.6) is straightforward. The covariance of the estimate is updated (Equation 3.7) using the Joseph stabilized form, to ensure the covariance remains positive semidefinite, even if the gain K_k is "suboptimal" [4].

The EKF is tuned using the weighting matrices Q_e and R_e, which represented the expected noise covariances of plant and measurement (see Section 3.3).

To note: both the prediction and the correction step can be performed multiple times, if (for example) a measurement is missed due to sensor fault, or respectively if multiple sensors are available and the measurements come in sequentially. Correcting sequentially can also be useful to avoid inverting large matrices (in Equation 3.5). As this rocket design uses multiple IMUs (each with their own y_{IMU}), the Correction is run multiple time sequentially, to include the encoder and each of the IMUs. All sensors have their own measurement model (and bias), Jacobian, and weighting.

Observability

To check if the filter can converge at a given state, the linearized system is examined for observability. The observability matrix (with the number of states $n = 13$) is

$$\mathcal{O} = \begin{bmatrix} H \\ HF_D \\ HF_D^2 \\ \vdots \\ HF_D^n \end{bmatrix} \tag{3.8}$$

[16] This is not the cleanest formulation, but hopefully understandable. See [4, pp. 259] for more thorough explanations of the algorithm.

which has rank 12 for the given model, i.e. the system is not fully observable. The reason is the lack of an absolute measurement in attitude, as the magnetometer measurement is invariant for rotations about the magnetic field vector. This will result in slow attitude drift from the true value, which is still an improvement over simply integrating the rate gyroscopes as the magnetometer constrains the attitude in the other two degrees of freedom.[17]

Initialization

The EKF is initialized by setting the state estimate and the covariance estimate to

$$\hat{x}(0) = \bar{x}_0 \qquad P(0) = \bar{P}_0 \tag{3.9}$$

where $\bar{x}_0$ can be computed while the rocket is stationary on the rail, and $\bar{P}_0$ is the uncertainty of the initial state (often quite low). Determining accurate initial values is treated in Section 3.6.

3.2 Optimal estimation theory

This section is here for completeness, it is not relevant for implementation. The system has continuous-time dynamics and discrete-time measurements [4]

$$\dot{x} = f(x, u, t) + w(t) \tag{3.10}$$

$$y_k = h(x_k, k) + v_k \tag{3.11}$$

where $w(t)$ is the process noise and v_k is the measurement noise, which are additive, uncorrelated, and white zero-mean Gaussian processes [4, 21]. Both processes satisfy [21] [18]

$$E\left[w(t)w^T(t+\tau)\right] = Q_e\delta(t) \qquad E\left[v_k v_{k+1}^T\right] = R_e\delta(k) \tag{3.12}$$

The estimation error $\varepsilon = x - \hat{x}$ is to be minimized in the cost function with weight vector ϱ [21]

$$\min V = \lim_{t\to\infty} E\left[\varepsilon^T \varrho \varrho^T \varepsilon\right] \tag{3.13}$$

for which the solution is of the form [21]

$$V^* = \varrho^T P \varrho \tag{3.14}$$

In infinite time, P satisfies the filter algebraic Ricatti equation [21]

$$PF + F^T P - PH^T R_e^{-1} HP + Q_e = 0 \tag{3.15}$$

This results in the estimator feedback [4]

$$\tilde{x} = K_k\left[y - h(\hat{x}_k)\right] \tag{3.16}$$

[17] For vehicles in straight and level movement, like ships, cars, and even airplanes, this is corrected by accelerometers measuring the gravity vector. However, the rocket will be in free fall the entire flight and measuring gravity is not possible using accelerometers.

[18] Prof. Werner please forgive the butchering of the theory that is to follow

where the gain K_k is computed with the a-priori covariance estimate P_k^-

$$K_k = P_k^- H_k^T \left[H_k P_k^- H_k^T + R_e \right]^{-1} \tag{3.17}$$

(since the posteriori covariance P_k is dependent on the current gain, the implicit equation would be $K_k = P_k^+ H_k^T R_e^{-1}$).

Due to the nonlinear behaviour of the state space functions, both P and K_k must be computed in real time as new measurements become available [4]. It is to note that the Kalman filter generally is only optimal for the mathematical model, not the actual plant (in this case the rocket) itself [4]. Additionally, the EKF is just quasi-optimal, as optimality is generally only proven for the case of linear state prediction and measurement functions.

3.3 Filter tuning

As seen in Equation 3.12, the tuning matrices Q_e and R_e represent the expected noise magnitudes of the state process and the measurements. Higher values in the matrices indicate a higher noise magnitude, i.e. lower trust in the accuracy of the measurement. The EKF is be tuned by adjusting the entries of these matrices, which must be positive semi-definite. The matrices can be filled to account for cross-covariance between the states or measurements (e.g. higher altitude or higher speed lowers accuracy of pressure measurement). However, this is often difficult to characterize, and the tuning matrices are often filled diagonally as initial guesses [20].

Typically, the values in R_e represent the noise behaviour of the sensors, where the the entries can be taken directly from sensor datasheets.

The matrix Q_e can be adjusted according to the expected “noise" in the state prediction process. Here, the entries can represent expected disturbances of the rocket flight, and again higher values indicate higher expected disturbances and thus lower trust in the accuracy of the state prediction.

Closed-loop stability depends on the robustness margins of the combined plant-estimator-controller loop. As shown in [22], using a state observer inside the control loop *could* lead to vanishing margins. This can lead to instability, if the plant models do not match reality perfectly. To recover some of the robustness margins of the optimal controller (see Section 2.3), a simple loop transfer recovery method is employed [20]

$$Q_e \quad \rightarrow \quad Q_e + \sigma B_e \tag{3.18}$$

where the weighting matrix Q_e has some added, fictitious noise σ acting on the control input channel. Here $B_e = \frac{\partial f}{\partial u}$ is the sensitivity of the input signal - but only for the actual control input, not for the accelerometer measurement (so B_e is all zero, except a single one in the δ row, δ column). In practice, this means that for higher weights in the δ/δ field of Q_e, the closed-loop robustness margins improve [20].

3.4 Response analysis

The performance of the EKF is evaluated by comparing the true and estimated states in simulation (for the simulation see Chapter II). The response for a simulated test flight are shown in Figure 3.1, where the true state x is compared to the estimated state $\hat{x}$. Both the body rates and the altitude (bottom row) are estimated very well, which is due

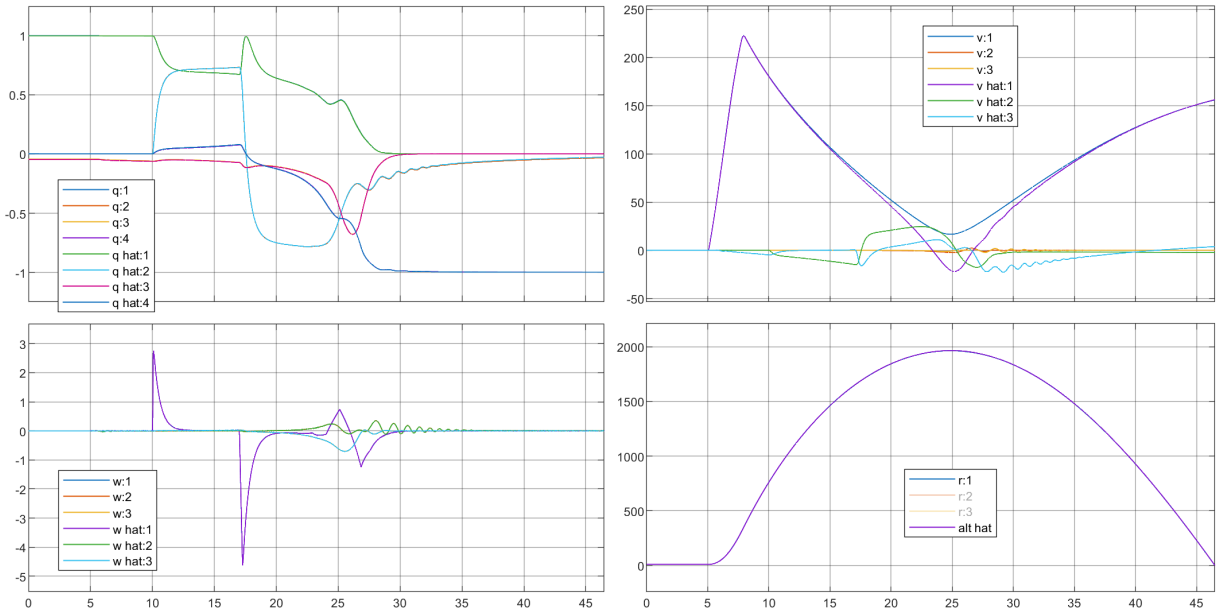


Figure 3.1: Simulation responses of the closed-loop estimated states for the simulated testflight. Rocket is performing a step roll program at t=10s, t=17s, and t=24s. Outputs: Quaternion attitude q, $\hat{q}$ (Top Left), Body rates ω, $\hat{\omega}$ [rad/s] (Bottom Left), Body velocity v, $\hat{v}$ [m/s] (Top Right), Altitude r_1, $\hat{alt}$ [m] (Bottom Right).

to the rate gyroscope measuring the state directly, and the barometer with an accurate atmosphere model having a direct coupling between measurement and state. The attitude quaternion (top left) is also estimated quite well due to having both rate integrals and the magnetic field measurements. The gyroscope bias is determined on the launch pad (see Section 3.6) which leads to low drift, and the magnetometer measurement further reduces perturbations from the true attitude. The estimate is not perfect however, as can be seen in a slight deviation q_2 between 25s and 35s.[19] The velocity estimate (to right) shows by far the largest deviation from the true values. There is no measurement available for the velocity, so this estimate relies largely on accelerometer and gyroscope data. While tuning, a strong coupling between velocity and attitude was noticed. As the rocket is essentially in free fall the entire flight, the accelerometer cannot measure the gravity vector, and any gravitational acceleration is modelled with the rocket attitude, see Equation 1.21. Also, as the rocket is oriented vertical, the forward velocity is the change in altitude ($v_x \approx \dot{alt}$), so there is a strong coupling of forward velocity to the barometer data / its estimated altitude. The velocity deviations become larger near apogee, where the rocket slows down and pitches over, before speeding up again.

Todo: insert responses of $\hat{C}_L$, $\hat{\delta}$

3.5 Implementation notes

Computation frequency

Computation time is an important factor in implementing the state estimation, as the discrete EKF process should be performed at $\omega_s \gtrapprox (20 \ldots 30) \cdot \omega_c$ [20] to avoid the rocket

[19] This .png makes it hard to see, the original plot has a higher resolution.

crossover region, which is in frequencies around $\omega_c \approx 5\text{Hz}$. The process frequency is chosen as $\omega_s = 200\text{Hz}$ to leave some margin for error.

Numerics

The Jacobians F and H are computed numerically. Numerical methods for finite differentiation are shown in Section A.1.1.

For the prediction step, numerical solvers are implemented to compute the solutions of the IVPs. Here, *explicit Euler* is used to compute a discrete state update. The methods are picked as a trade-off between accuracy and computational effort.

Quaternions

To ensure valid attitude quaternion operations, the norm of the quaternion has to be 1. To ensure this unit norm, the quaternion is normalized after every operation changing its values, as $q = q/\text{norm}(q)$ in the prediction model and after the correction step $\hat{x}_k^+ = \hat{x}_k^- + K_k \left[y_k - h(\hat{x}_k^-)\right]$, respectively.

Projection

As the controller expects not the full state x, but the roll state x_R and the flight condition x_{FC}, some post-processing has to be performed. This includes transforming the quaternion attitude to the roll angle, and computing the dynamic pressure from velocity and air data.

3.6 Pad filter

This algorithm serves to compute accurate values for $\bar{x}_0$, and the sensor biases b. While the rocket is stationary on the launch rail, the Pad filter runs in the Estimator loop instead of the EKF. Shortly before liftoff, the Pad filter hands over the latest state estimate ($\bar{x}_0$) and the determined sensor biases to the EKF. To get accurate values for the sensor biases, the Pad filter runs a lowpass filter with all the current sensor data z_k and the already filtered data s_k (start with $s_0 = \mathbf{0}$)

$$s_{k+1} = \alpha z_k + (1 - \alpha) s_k \tag{3.19}$$

This should take place over a long time frame (longer than a minute) to attenuate any noise effects, or movement through plumbing or wind. The gain α is chosen according to the allowable/expected time frame of initialization, a lower α results in a smoother, but slower converging signal (α is the time constant of this exponential discrete-time filter, the resulting shape is $e^{\alpha\, dt}$ with sampling time dt).

The IMUs should also be at a constant temperature, which shall[20] be the same as in flight (though some IMUs perform internal correction of the gyroscope temperature). The biases and earth magnetic field are combined to the bias vector b.

As the filter runs while the rocket is stationary on the launch rail, several assumption can be made:

1. The velocity of the rocket v will be zero, and thus the altitude l is constant.

[20] Or rather should, as this is not enforceable.

2. The acceleration of the rocket $\dot{v}$ will be zero.
3. The body rates ω will be zero, and thus the attitude q is constant.

With the filtered data s, the state can be computed using the sensor models as follows:

1. The accelerometer measurement will be the rotated gravity vector, so $S_A A = S^T g^G$. No bias can be set for the individual axes, but the magnitude of the accelerometer measurement (part of s_k) should be g.
2. The gyroscope bias b_Ω can be set to the filtered angular velocity (part of s_k) while the rocket is stationary.
3. Altitude can be computed using the filtered pressure (part of s_k), according to the (inverted) atmosphere model. As the launch altitude is is known, any bias offset of the barometer b_P can be corrected for.
4. The initial attitude will be close to, but not exactly upright. It is defined as yaw/pitch only, and thus can be computed uniquely from the accelerometer measurement.
5. The magnetic field vector of the Earth M_E^G can be computed from the filtered magnetic measurement M (part of s_k) and by inverting the estimated initial attitude.

3.7 Hard calibration

The orientation of all sensor frames (S_Ω, S_A, S_M) to the rocket body must be determined *well before* launch day. Additionally, the position of the accelerometers d_A to the dry center of gravity, and the the hard (b_{HI}) and soft (B_{SI}) iron biases for the magnetometer correction should be determined.

Chapter II

Rocket Flight Simulation

Joe Dolina

4 Background

4.1 Borealis Airbrakes

The rocket Borealis (for the 2024 LC competition) was Waterloo Rocketry's first attempt at flying (and for all intents and purposes, developing) active control systems, in the form of airbrakes. At the outset of the airbrakes project, it was envisioned that a working group would develop something like Figure 4.1.

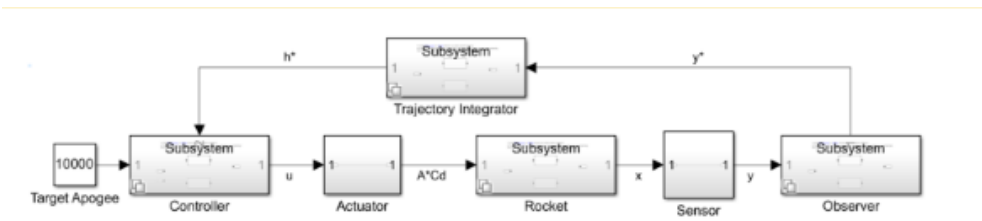


Figure 4.1: Airbrakes control loop structure

Everything except "Actuator" and "Rocket" were implemented to some degree in the final flight software, but Simulink was never actually used other than to create the above diagram.

A PID controller was ultimately selected for airbrakes because the team lacked the knowledge and scope for anything more complex. This controller was "tuned" using a plugin for OpenRocket (OR) [23], dynamically modifying the drag force used by OR in its calculations with values produced from external computational fluid dynamics simulations. Later versions included the ability to inject random noise into the state measurements. Under these noisy conditions, simulations suggested the controller could get within about 20m of the target altitude over most of the controllable regime (20,000 +/- 2,000 ft.).

While much more useful at the time than having no controller-in-loop simulation options at all, the OR plugin was not sustainable. The overlap between Waterloo Rocketry members (and budding control engineers) and Java developers was and still is very small. The final plugin version ran the controller as a binary compiled from the same C source code as the board on the rocket, and while this was also a promising development in the team's testing practices, the process for doing this was even less accessible than running the plugin itself. The plugin project was ultimately an uphill battle to turn OR into something it

wasn't meant to be. This motivated the development of a complete simulation alternative in Simulink which could be effectively used for estimator/controller development and validation.

4.2 Project Jupiter

Projeto Jupiter is a rocketry team from the University of São Paulo, Brazil. They conveniently already developed most of what is necessary for a plant model of a sounding rocket with canards in Simulink [24]. Rather than trying to start more or less from scratch as it was planned last year, the goal this year was essentially to modify their plant model with our rocket design parameters, verify it for uncontrolled flight against the OR baseline, and then begin simulating the effects of canards. Although the overall structure of the model was adopted, the actual implemented calculations were either novel or ended up heavily modified.

5 Theory

Compared to a dynamic model of a real launch vehicle, a sounding rocket is considerably simpler. In particular, sloshing, bending, and TVC dynamics do not need to be considered. The rocket can be treated as a single rigid body with time-varying mass and mass distribution. The only forces acting on an uncontrolled rocket are engine thrust, gravity, and drag. Of these, gravity applies no moment by definition, and the moment applied by thrust, due to misalignment with the rocket centre of mass, is constant and typically negligible. Since gravity is well defined and engine thrust is defined empirically via propulsion test data, this leaves the aerodynamic forces and moments as the most complicated aspect of simulation.

The general equation for aerodynamic forces and moments is

$$F = \frac{1}{2}\rho V_A^2 C_i(\mathrm{M}, \alpha, \omega) A_r \tag{5.1}$$

$$M = F \times d \tag{5.2}$$

with the atmospheric density from the standard atmosphere model ρ, the local air velocity including wind disturbances V_A, and the reference area (e.g. body cross section) A_r. The moment is a cross product of the net force vector F and the vector from the rocket centre of gravity to the centre of pressure d. The coefficient C_i (indexed by the force to be calculated) is an explicit function of the Mach number M, the angle of attack α, and the rockets angular velocity ω. It depends implicitly on the Reynolds number, on the geometry, surface, and size of the rocket – in short, the calculation of the coefficients is complicated.

Barrowman [12] provides empirical methods for computing the centre of pressure, and the aerodynamic coefficients and derivatives of sounding rockets (for our relevant Mach numbers). The method considers the major components (i.e. fins, body nose, etc) of the rocket separately, which simplifies computing and allows easy configuration of parametrized components in implementations (e.g. OpenRocket [14, 25]). The characteristic aerodynamic parameters to be calculated are the centres of pressure, normal forcing and picth damping coefficient derivatives, the roll forcing and damping coefficient derivatives, and the drag coefficient [12]. According to Barrowman, the method predicts the aerodynamic characteristics within 10% of wind tunnel and flight data [12].

The only significant change from an aerodynamics standpoint is the Prandtl factor $\mathcal{B}$, which "corrects" the coefficients depending on the Mach number. Barrowman assumes the Prandtl term to be

$$\mathcal{B}^{-1} = \frac{1}{\sqrt{|1 - \mathrm{M}^2|}} \tag{5.3}$$

which has a singularity at $\mathrm{M} = 1$. However, the real coefficient does not approach infinity, and both wind tunnel and flight data show that the Prandtl factor peaks at somewhere around 2 or 3 near sonic speeds [11]. The implemented correction thus takes $\mathcal{B}^{-\infty}$ to be the minimum of 5.3 and 2.

The coefficient of drag (C_d) is modelled only as a function of Mach number for a given rocket, using either OR or RASAero data. In the future, other drag models may be considered.

ToDo: link flight dynamics docs

6 Implementation

The simulation is implemented in Matlab Simulink. In order to have some semblance of version control (as Simulink `.slx` files are binary) and to keep the simulation maintainable, the model is in a hierarchical modular architecture.

Each major component of the whole plant, such as flight dynamics, actuators, environmental conditions, or sensor emulation, is encapsulated in its own subsystem. The major subsystems are then further structured into layers, e.g. flight dynamics – forces – normal forces – body component lift. Each layer computes a specific aspect of the physical model, aggregating contributions from lower-level components. This improves traceability, and makes debugging easier by isolating the influence of individual calculations (e.g., lift of fins, body tube, canards). This modularization eases development and testing of individual components, as intermediate results (e.g. local force contributions) can be verified against theoretical values or simulation baselines. The architecture also allows to substitute or refine specific models (e.g. replacing the canard moment calculation with a more detailed version) without disturbing the overall system architecture.

The model made extensive use of Matlabs' *Aerospace Blockset* (AB), which has ready-to-use implementation blocks for a lot of needed components, e.g. equations of motion, geodetic and magnetic models, IMUs, atmosphere models and accurate wind disturbances, etc. All units are SI - MKS, to maintain consistency across subsystems and minimize the need for unit conversions.

6.1 Simulation Setup

The simulation is initialized and parameterized via a configuration script, where the user selects the rocket script to be used.

A rocket-specific script (e.g. `borealis.m`) imports a `.csv` file from an OpenRocket export, containing relevant trajectory and performance data such as time series of thrust, mass, and aerodynamic coefficients. The OR wind model has not been investigated, and so all OR data is supplied for the no wind case. Since OpenRocket does not provide complete geometric metadata, this script also explicitly defines additional parameters such as

reference dimensions, component locations, mass and moment of inertia. Another sensor-specific script sets values for sensor parameters such as noise density, bias stability, and limits. These values are extracted from sensor datasheets or testing.

The configuration script also performs common post-processing on the data as necessary, including numerical differentiation of the time-dependent mass to calculate the mass flow rate, and converting the C_d/time and Mach/time series into a direct C_d/Mach lookup table. This transforms the OR data into the format the simulation expects. This modular design allows for rapid integration of new vehicle configurations, provided OpenRocket geometry and performance exports are available.

6.2 Top Level Plant

The plant model is composed of three subsystems: the canards actuator, the rocket flight dynamics, and the sensors. Figure 6.1 shows the connection of these subsystems. A visualization subsystem plots the output values of the flight dynamics against the OR results for validation purposes.

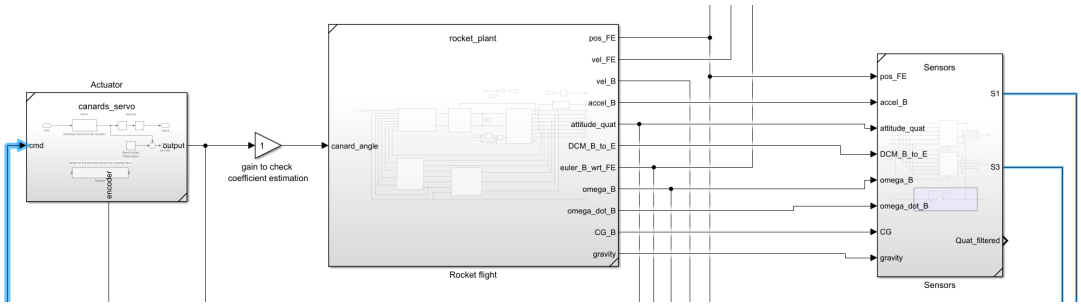


Figure 6.1: Rocket plant model interface, with simulated rocket flight (centre), actuator dynamics (left), and sensor models (right).

6.3 Actuator

The servo is modelled as a nonlinear second-order system, which includes position and rate limits. To account for gearing between the servo and the canards, a deadband and hysteresis are used to simulate internal backlash. This is in contrast to the linear first-order model used by the estimator. To measure the actual canard angle, the actuator is equipped with a rotary potentiometer mounted to one of the canard shafts. The potentiometer is modelled with additional white noise.

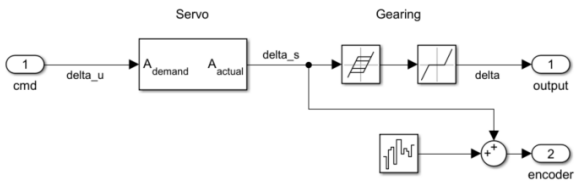


Figure 6.2: Canards Actuator Dynamics

6.4 Sensors

In order to test the robustness of the controller to real-world sensor outputs, the IMU (accelerometer, gyroscope), magnetometer, and barometer (only pressure) outputs are simulated, adding white noise and other sensor effects (shown in Fig. 6.3). The accelerometer and gyroscope are modelled with AB's IMU block, which accounts for sensor eigendynamics, limits, noise, bias, and (cross-axis) scaling. The magnetometer uses the International Geomagnetic Reference Field, which is implemented in the AB. The magnetic field is rotated to the body frame, and noise, scaling, and bias are added to the signal. The barometer is modelled with the International Standard Atmosphere block from the AB, and again noise, bias, and scaling is added to the signal.

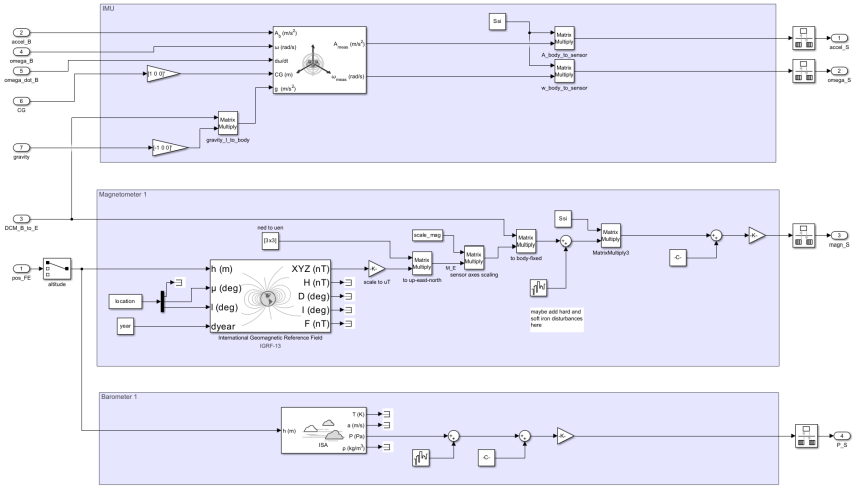
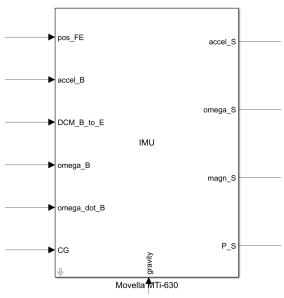
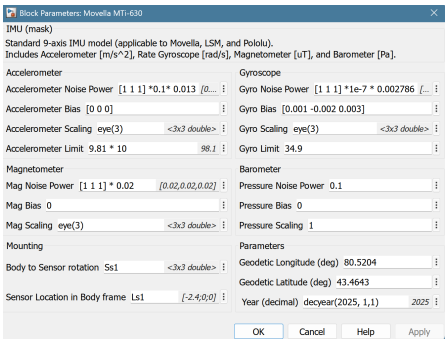


Figure 6.3: Sensor simulation

The sensor simulation is combined to a custom Standard IMU block, which has a mask to input all sensor parameters (shown in Fig. 6.4). The values of those parameters are taken from datasheet values, and may be adjusted based upon sensor testing. The estimator is connected to 3 unlike IMUs for redundancy, and to alleviate saturation. The Movella MTi-630 is an integrated Altitude and Heading Reference System (AHRS), which contains its own onboard orientation filter. The LSM6DSV32X and AltIMU-10 are an IMU and IMU carrier board respectively. They are henceforth all referred to as "IMUs". In the final implementation, data is sampled from all IMUs at the nominal execution rate, concatenated, and passed to the estimator for processing.



(a) Standard IMU component block



(b) IMU instance parameter interface

Figure 6.4: Standard IMU component instance

6.5 Rocket Flight

All time-based look-up tables (e.g. thrust, mass, centre of gravity) with OR data are connected to a global clock. To simulate idle time on the launch rail, a user-defined idle duration can be specified via the initialization script. This offsets the simulation time prior to OpenRocket's nominal $t = 0$, effectively holding the rocket in a static state (with rail constraints) before powered ascent begins. This is a necessity for accurately modelling the pre-launch controller initialization.

6DOF Variant Mass Model

The core of the rocket flight dynamics model is the "Simple Variable Mass 6DOF" block by the Aerospace Blockset, providing equations of motion in six degrees of freedom. It takes in net force and moment vectors about the rocket centre of mass, and outputs the kinematic states of position, velocity, acceleration, orientation, angular velocity and acceleration. Note that the inputs are provided in the body fixed frame, with the X-axis coincident with the longitudinal axis of the rocket body. The translational position and velocity are provided in the flat earth reference frame. The origin of this inertial frame is arbitrary for dynamics purposes, but is effectively the position of the launch rail on the earth surface. The 6DOF block does not output mass vs time, instead providing inputs for starting mass, ending mass, and mass flow rate $\frac{dm}{dt}$. The moment of inertia is interpolated linearly between initial and final moments of inertia, using the mass flow rate $\frac{dm}{dt}$.

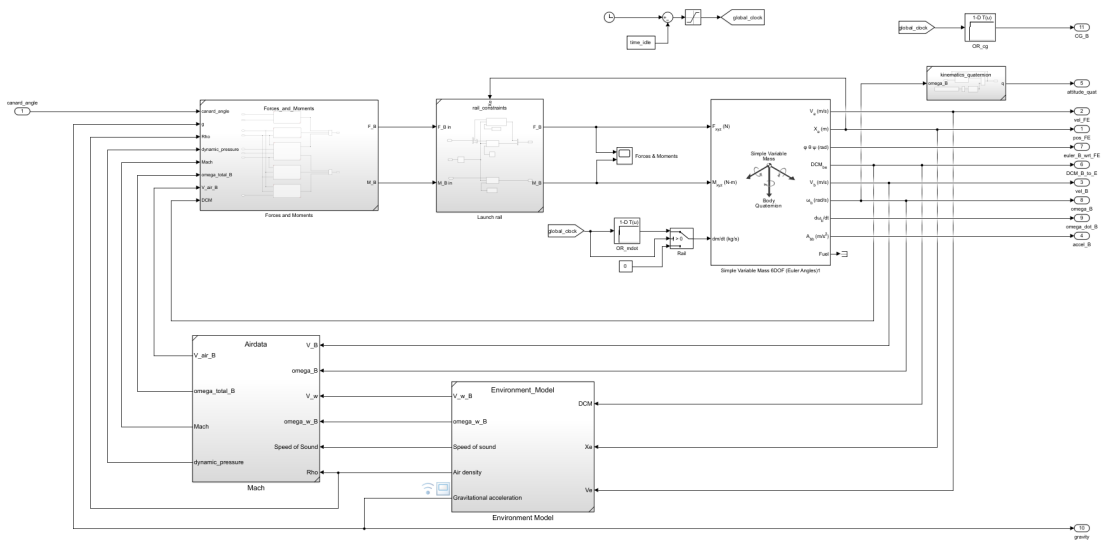


Figure 6.5: Rocket flight system, with 6DOF Equations of Motion, wind disturbances, external forces and moments, rail constraints.

Environment, Air data

To accurately model gravitation and the surrounding air, environment blocks from the AB are used. The WGS84 Earth model is used where applicable for geodetic reference, such as coordinate transformations and gravitational acceleration. The air is modelled using the International Standard Atmosphere, which computes air density and the local speed of sound according to the current altitude.

The Airdata subsystem calculates the rocket velocity relative to the oncoming air V_A,

adding the velocity relative to the Earth and the local wind velocity vector. Using the environment data, the Mach number and dynamic pressure ($0.5\rho V_A^2$) are calculated.

Wind disturbances

The wind environment is modelled as a layered system combining steady-state wind, stochastic turbulence, and discrete gust events. The steady-state wind and stochastic turbulence operate similar to OpenRocket [14]. The baseline wind assumes a constant wind velocity over all altitudes. While this assumption simplifies the vertical wind profile, it remains sufficient in the absence of detailed meteorological data. Superimposed on the steady wind is a stochastic turbulence using pink noise (with higher noise amplitude at lower frequencies, 1/f) This introduces realistic low-frequency variability to the wind vector, creating stochastic lateral and vertical disturbances. This stochastic turbulence is also responsible for all rotational components of the wind pertubations.

Additionally, discrete gusting is added to the baseline and stochastic winds, which allows the user to define transient changes in wind velocity. Each gust is specified by magnitude, direction, time of onset and duration.

Rail constraints

While the rocket remains on the launch rail, its motion is constrained to a single (or rather half a) degree of freedom in the rocket forward direction. This models the effect of launch lug constraints, which restrict lateral and angular motion prior to rail clearance to keep the rocket vertical (until asufficient vertical velocity is achieved, to prevent wind-induced pitch over).

This is implemented by filtering out all forces and moment but the positive forces in the x-direction, while the rocket is under a certain altitude – in this case the start altitude, plus the length of the launch rail. This effectively locks the vehicle in place until liftoff and prevents any unrealistic drift or rotation while idle on the rail. It also ensures that simulation of rail clearance behavior and initial angular acceleration are accurate.

6.6 Forces and Moments

The total force and moment vectors acting on the rocket are computed by summing contributions from gravity, propulsion, and aerodynamics, as shown in Fig. 6.6. These forces and moments are computed in the rocket's body-fixed coordinate frame, relative to the instantaneous centre of gravity.

Propulsion

The propulsion force is modelled using a time-series look-up table, indexed by the global clock. This thrust curve provides instantaneous thrust at discrete time points. It is imported from a preprocessed OR export (or directly from static fire test data).

The thrust vector is assumed to align with the rocket longitudinal axis (body-x), and thus produces no moment about the rocket centre of gravity. Gimballing or off-axis thrust components are currently not modelled, but could be added later on.

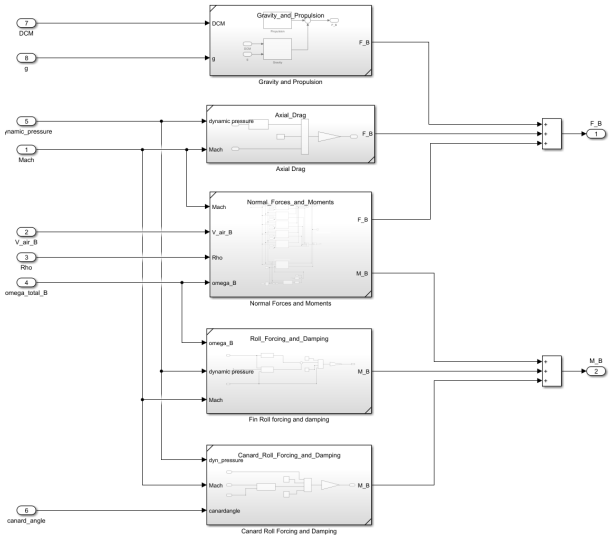


Figure 6.6: External forces and moments

Aerodynamic Drag force

Aerodynamic drag is implemented via a C_d/Mach look-up table, generated from OR data. This table provides the drag coefficient C_d as a function of Mach number, based on vehicle geometry defined in OR.

The drag force is computed with this C_s and the dynamic pressure, as it is assumed the vector of oncoming air is mostly in the x-direction. The drag is

$$F_D = 0.5\rho V_A^2 C_d A_r \tag{6.1}$$

with the atmospheric density ρ, the local air velocity V_A, and the body cross section A_r.

Corrections for angle of attack and transonic behavior are not yet implemented in this module but may be included later.

Aerodynamic Roll torque

The roll dynamics of the rocket are driven primarily by the aerodynamic forces generated by the fins and canards. For flexibility they are split into separate components and treated independently. This allows the canards to be handled with more detail or with different aerodynamic models if needed, and to easier accommodate different flight regimes or experimental configurations.

In the contribution of the fins, the aerodynamic coefficients and variable derivatives are computed in pre-processing using Barrowman's equations [12], and then fed into the Simulink model. The Barrowman method takes into account the fin's aerodynamic area, span, mean chord, and the angle of attack to compute the moment coefficients. The derivatives are used by the "roll forcing and damping block" in the plant model, the forcing component is shown in Fig. 6.7.

While the fins are treated using Barrowman's equations, the canard contribution to roll dynamics are handled slightly differently to correspond to aerodynamic development done by the Flight Dynamics subsystem [13]. The coefficient of the canards is computed using thin airfoil theory for low aspect delta wings with supersonic leading edges, found in Stengel [11, p. 73].

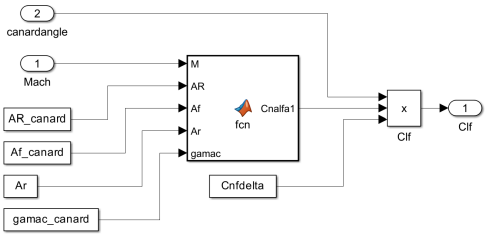


Figure 6.7: Canard roll torque, roll forcing component

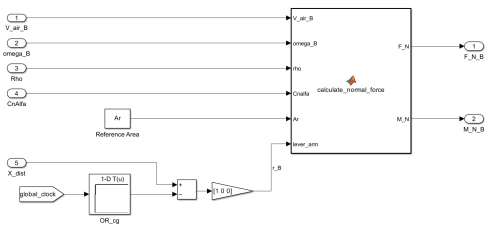
Aerodynamic Normal forces and Moments

The methodology used to compute the normal force components, which affect pitch dynamics of the rocket, is the same as that of OpenRocket [25, sec. 3.2.1]. The rocket body is divided into its major components: nosecone, body, fin, tail, and canards. Each component is treated as a separate aerodynamic entity, and the normal force and moment contributions from each are computed individually based on local airflow conditions. The local velocity of oncoming air at the component centre of pressure is computed to calculate the normal force and moment vectors, along with the normal force coefficient derivative C_{N_α}. Each component's contribution to the normal force vector is given by:

$$F_N = 0.5\rho V_A^2 C_{N_\alpha} \alpha A_r \qquad\qquad M_N = F_N \times d \tag{6.2}$$

This functionality is encapsulated in a generic "lift component" block, whose internals are shown in Fig. 6.8. The individual component forces and moment vectors are added to produce a net aerodynamic normal force and pitch moment.

This approach is integral to predicting the passive aerodynamic stability of the rocket, the rocket is passively stable if the the net moment produced by any angle of attack is driving the angle of attack to zero – i.e. the rocket is stable if it turns into the wind.



(a) Lift component block structure

```
function [F_N, M_N] = calculate_normal_force(V_air_B, omega_B, rho, Cnalfa, Ar, lever_arm)
    % relative velocity
    Vrel = V_air_B + cross(omega_B, lever_arm);

    % Body relative velocity magnitude
    Vrel_mag = norm(Vrel);
    VrelMagYZ = norm(Vrel(2:3));

    if VrelMagYZ ~= 0 % prevent division by 0
        % Angle of attack and component separation
        attack_angle = asin(VrelMagYZ/Vrel_mag);
        F_N_dir = [0; Vrel(2) / VrelMagYZ; Vrel(3) / VrelMagYZ];

        % Calculating total force
        F_N_mag = 0.5 * rho * Vrel_mag^2 * Ar * Cnalfa * attack_angle;
        F_N = F_N_dir * F_N_mag;
    else
        F_N = [0; 0; 0];
    end
    % Total torque
    M_N = cross(F_N, lever_arm);
end
```

(b) Lift component calculation

Figure 6.8: Lift component subsystem

7 Plant simulation results

7.1 Comparison with OpenRocket

The plant simulation results are compared against the predictions from OpenRocket, focusing on several flight parameters. The comparisons were conducted under nominal conditions, with the simulation run for a flight trajectory without wind disturbances. Even though the OR wind model is layed out in Niskanen [14], the specific values for the stochastic turbulence are difficult to reproduce.

One simulation result from a simulated Borealis flight is shown in Fig. 7.1. The horizontal axis represents time in seconds, and each subplot displays the time evolution of one of the flight parameters. Note that the OR simulation deploys a parachute at apogee, which this plant model does not.

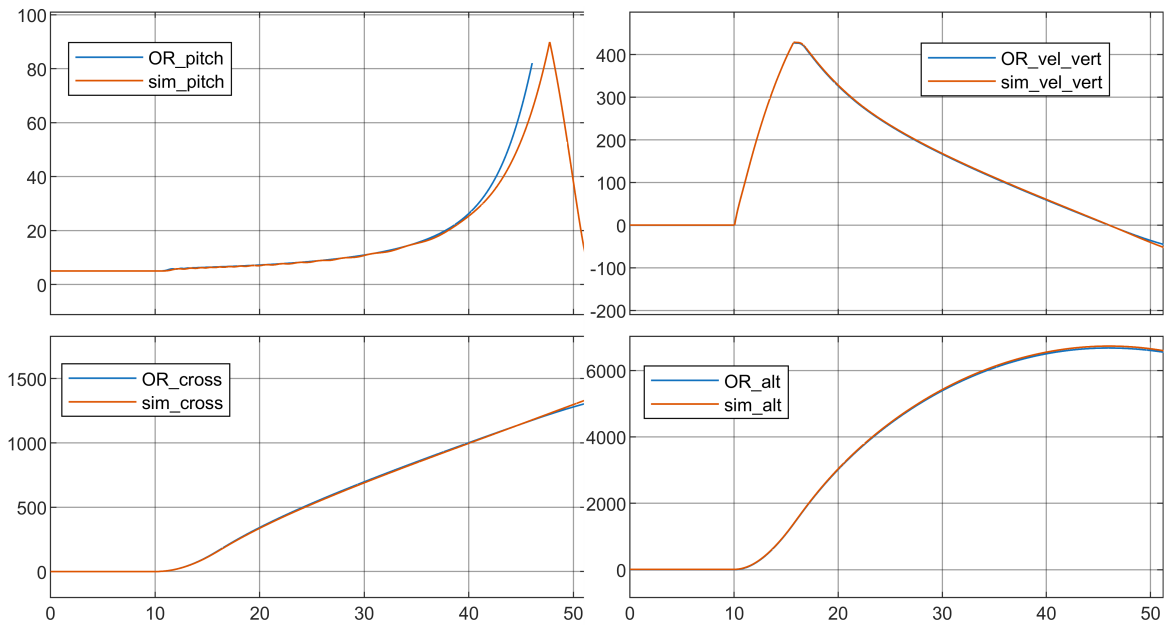


Figure 7.1: Simulation results (red) compared with OpenRocket (blue), Borealis nominal flight. Common abscissa is time [s]. (Top Left) Pitch angle magnitude [deg], (Top Right) Vertical velocity [m/s], (Bottom Left) Cross range distance [m], (Bottom Right) Altitude [m].

The pitch angle trajectory from the simulation shows a similar trend to the OpenRocket results, indicating that the simulation captures the overall pitch dynamics of the rocket. However, the pitch angle in OR increases faster than this plant model, leading to a large deviation close to apogee. Additionally, a pitch oscillation occurs shortly after liftoff, where OR shows a faster response than this model. Both phenomena could be explained by significant differences in the pitch damping computation.

Both the simulation and OpenRocket predict a nearly identical profile for vertical velocity. This is due to the shared thrust curve between the models, and that the plant pre-processing creates an accurate C_d/Mach look-up table from the OR export. As the pitch angle is very similar during powered ascent (10s until 15s), the thrust acts in the same proportion to the vertical acceleration.

The cross-range distance, which measures the lateral displacement of the rocket from its launch point, is also very similar. As no wind disturbances were added, this is to be expected.

The altitude profiles from both the simulation and OpenRocket are almost identical during ascent, but diverge slightly near apogee. This is due to a small accumulated difference in the vertical velocity.

Overall, the simulation shows good agreement with OpenRocket across all measured parameters, validating the accuracy of the plant model under the conditions tested.

Chapter III

Simulation and Flight Results

8 Simulation

8.1 Estimator in the loop

8.2 Controller in the loop

8.3 Full closed-loop

9 Hardware in the Loop

10 Testflight

11 Competition flight

References

[1] Waterloo Rocketry. 2025. URL: https://www.waterloorocketry.com.

[2] Launch Canada. 2025. URL: https://www.launchcanada.org/lc-competition-2025.

[3] Peter H. Zipfel. *Modeling and Simulation of Aerospace Vehicle Dynamics*. 2nd. Reston: American Institute of Aeronautics and Astronautics, 2007. ISBN: 978-1-56347-875-8. DOI: 10.2514/4.862182.

[4] Frank L. Lewis, Lihua Xie, and Dan Popa. *Optimal and Robust Estimation*. 2nd. CRC Press, Taylor & Francis Group, 2008. ISBN: 978-0-8493-9008-1.

[5] Brian L. Stevens, Eric N. Johnson, and Frank L. Lewis. *Aircraft Control and Simulation: Dynamics, Controls Design, and Autonomous Systems*. Third edition. Hoboken, New Jersey: Wiley, 2015. ISBN: 9781119174882. DOI: 10.1002/9781119174882.

[6] George M. Siouris. *Missile Guidance and Control Systems*. New York, NY: Springer, 2004. ISBN: 978-0-387-00726-7. DOI: 10.1007/b97614.

[7] Joan Solà. *Quaternion kinematics for the error-state Kalman filter*. Technical University of Catalonia, Institut de Robòtica i Informàtica Industrial, 2017.

[8] Werner Schiehlen and Peter Eberhard. *Technische Dynamik*. 5th. Springer Vieweg, 2017. ISBN: 978-3-658-18456-8. DOI: 10.1007/978-3-658-18457-5.

[9] *VectorNav Inertial Navigation Primer*. 2024. URL: https://www.vectornav.com/resources/inertial-navigation-primer/ (visited on 12/07/2024).

[10] United States Committee on Extension to the Standard Atmosphere. *U.S. Standard Atmosphere, 1976*. NOAA Document S/T 76-1562. Washington, D.C.: National Oceanic and Atmospheric Administration (NOAA), 1976. URL: https://ntrs.nasa.gov/citations/19770009539.

[11] Robert F. Stengel. *Flight Dynamics*. Princeton, NJ: Princeton University Press, 2004. ISBN: 9781400866816. DOI: 10.1515/9781400866816.

[12] James S. Barrowman. "The practical calculation of the aerodynamic characteristics of slender finned vehicles". M.Sc. thesis. USA – Washington, D.C.: The Catholic University of America, 1967. URL: https://ntrs.nasa.gov/api/citations/20010047838/downloads/20010047838.pdf.

[13] Luca Scavone. *Canard aerodynamics*. Waterloo Rocketry, Flight dynamics, 2025.

[14] Sampo Niskanen. "Development of an Open Source model rocket simulation software". M.Sc. thesis. Finland – Helsinki: Helsinki University of Technology, 2009. URL: https://openrocket.sourceforge.net/thesis.pdf.

[15] Ben Pickens. *Canards mechanical design*. Waterloo Rocketry, Controls, 2025.

[16] Ashish Tewari. *Advanced control of aircraft, spacecraft and rockets*. Aerospace series. Hoboken, N.J: Wiley, 2011. ISBN: 1119971209. DOI: 10.1002/9781119971191.

[17] Julian Theis. *Flight Control Law Design: Lecture Notes*. Hamburg University of Technology, Institute of Aircraft Systems (FST), 2023.

[18] Bryan Tong Minh. "Real-time position and attitude determination of the Stratos II sounding rocket". M.Sc. thesis. Delft University of Technology, 2012.

[19] Julian Theis et al. "Linear Parameter-Varying Feedforward Control: A Missile Autopilot Design". In: *AIAA Guidance, Navigation, and Control Conference 2015*. Red Hook, NY: Curran, 2015. ISBN: 978-1-62410-339-1. DOI: 10.2514/6.2015-2001.

[20] Herbert Werner. *Control Systems Theory and Design: Lecture Notes*. Hamburg University of Technology, Institute of Control Systems, 2021.

[21] Herbert Werner. *Optimal and Robust Control: Lecture Notes*. Hamburg University of Technology, Institute of Control Systems, 2021.

[22] J. Doyle. "Guaranteed margins for LQG regulators". In: *IEEE Transactions on Automatic Control* 23.4 (1978), pp. 756–757. ISSN: 0018-9286. DOI: 10.1109/TAC.1978.1101812.

[23] *OR Airbrake Plugin*. https://github.com/waterloo-rocketry/or-airbrake-plugin.

[24] *Project Jupiter Control Canards*. https://github.com/Projeto-Jupiter/CC.

[25] *OpenRocket technical documentation v13.05*. https://openrocket.sourceforge.net/techdoc.pdf.

[26] Joaquim R. R. A. Martins and Andrew Ning. *Engineering Design Optimization*. Cambridge University Press, 2021. ISBN: 9781108833417.

[27] Joaquim R. R. A. Martins, Peter Sturdza, and Juan J. Alonso. "The complex-step derivative approximation". In: *ACM Trans. Math. Softw.* 29.3 (Sept. 2003), pp. 245–262. ISSN: 0098-3500. DOI: 10.1145/838250.838251.

[28] David Griffiths and Desmond Higham. *Numerical Methods for Ordinary Differential Equations: Initial Value Problems*. Jan. 2010. ISBN: 978-0-85729-147-9. DOI: 10.1007/978-0-85729-148-6.

Appendix A

Design appendix

A.1 Numerical methods

A.1.1 Numerical Jacobian computation

The Jacobian J_x is the matrix of all first-order partial derivatives of a vector-valued function. For the prediction and measurement model, the Jacobians of the functions f and h [1] are needed with respect to the state x (where $n = \text{length}(x)$). The Jacobian of any function with respect to x is defined as

$$J_x = \frac{\partial f}{\partial x} = \begin{bmatrix} \frac{\partial f_1}{\partial x_1} & \cdots & \frac{\partial f_1}{\partial x_n} \\ \vdots & \ddots & \vdots \\ \frac{\partial f_m}{\partial x_1} & \cdots & \frac{\partial f_m}{\partial x_n} \end{bmatrix} \tag{A.1.1}$$

Jacobians may be derived analytically from the function, but this is often very cumbersome. Therefore, it is common practice to compute the matrix numerically. A frequently used method is using *finite differences*, however this is known to be not very efficient nor accurate, and has issues with numerical instability [26].

Complex-step differentiation

A more efficient way to calculate Jacobian matrices numerically is to use *complex step differentiation*. In this method with step size ε, the k-th column of the matrix (J_{x_k}) is [27]

$$J_{x_k} = \text{Im}\left(\frac{f(t, x + \mathrm{i}\varepsilon e_k, u)}{\varepsilon}\right) \tag{A.1.2}$$

where e_k is the k-th unit vector in the state-space (i.e. $e \in X$, and $e_1 = \begin{bmatrix} 1 & 0 & \cdots & 0 \end{bmatrix}^T$, $e_2 = \begin{bmatrix} 0 & 1 & \cdots & 0 \end{bmatrix}^T$, etc.). This must be computed for each entry of x, so that the Jacobian is

$$J_x = \begin{bmatrix} J_{x_1} & J_{x_2} & \cdots & J_{x_n} \end{bmatrix} \tag{A.1.3}$$

The evaluation of the functions with a complex state vector $\begin{bmatrix} x_1 + \mathrm{i} & x_2 & \cdots & x_n \end{bmatrix}^T$ seems very unintuitive, but works for every holomorphic $\mathbb{R}^n \to \mathbb{R}^m$ function[2] [26]. Complex-step differentiation also is numerically stable, and has also an $\mathrm{O}(h^2)$ error size [27]. Both finite differences and complex-step scale linearly with the number of variables, but the finite difference method requires two function evaluations per variable, while complex-step requires only one.

[1] The equations are shown for an "f", but are of course the same for f and h

[2] for implementation this requires that the function can be evaluated with complex inputs

A.1.2 Numerical Integration methods

To get a new discrete state for the prediction process of the EKF from the continuous-time model, the nonlinear differential equations

$$\dot{x}(t) = f(t, x, u) \qquad x(0) = x_n \tag{A.1.4}$$

have to be solved. As the last state estimate is available as an initial value, this can be expressed as an *Initial Value Problem* (IVP). The IVP can be solved by numerical integration, see Equation 3.1

$$\hat{x}_k^- = \underbrace{\int_{t-dt}^{t} \dot{\hat{x}}(t, x, u) d\tau}_{f_D(x,u)} \qquad \hat{x}(t - dt) = \hat{x}_{k-1}^+ \tag{A.1.5}$$

To minimize computational effort the integration methods should be explicit, as the filter in the prediction step has only past state information. An implicit method would require (potentially unstable) root finding methods, which are computationally expensive[3].

Explicit Runge-Kutta method

The explicit s-stage *Runge-Kutta* method is compactly written as [28]

$$x_{n+1} = x_n + \varepsilon \sum_{i=1}^{s} c_i k_i \tag{A.1.6}$$

$$k_i = f\left(t_n + a_i \varepsilon\,,\ x_n + \varepsilon \sum_{j=1}^{i-1} b_{ij} k_j\,,\ u\right) \tag{A.1.7}$$

with the time step size ε, which in case of the EKF is the sampling time (i.e. the time since the last filter iteration). The constants a_i, b_{ij}, and c_i are the entries of the vectors a and c and the matrix B, displayed in a *Butcher tableau*

$$\begin{array}{c|c} a & B \\ \hline & c^T \end{array} \tag{A.1.8}$$

These can be very large, as in the *Dormand-Prince* method (RK45, which is the standard solver in Matlab). For our cases we will keep the tableaus small. The simplest, first order ($s = 1$) method is explicit Euler, with the butcher tableau [28]

$$\begin{array}{c|c} 0 & 0 \\ \hline & 1 \end{array} \tag{A.1.9}$$

which results in the simple formula

$$x_{n+1} = x_n + \varepsilon f\left(t_n,\ x_n,\ u_n\right) \tag{A.1.10}$$

Other integration methods can be formulated similarly, for example the discrete update function of the quaternion kinematics[4] in Equation 1.12. The second order ($s = 2$) butcher tableau of the Heun method [28] is

$$\begin{array}{c|cc} 0 & & \\ 1 & 1 & \\ \hline & \frac{1}{2} & \frac{1}{2} \end{array} \tag{A.1.11}$$

[3] Inverting the Jacobian in Newtons method

[4] As unit quaternions are used for attitude description, they have to be either normed after each Euler integration, or the discrete update has to follow their geometric structure, as shown in the Equation above.

The fourth order ($s = 4$) butcher tableau of the classical Runge-Kutta method RK4 [28] is

$$\begin{array}{c|cccc} 0 & & & & \\ \frac{1}{2} & \frac{1}{2} & & & \\ \frac{1}{2} & 0 & \frac{1}{2} & & \\ 1 & 0 & 0 & 1 & \\ \hline & \frac{1}{6} & \frac{1}{3} & \frac{1}{3} & \frac{1}{6} \end{array} \tag{A.1.12}$$

A.2 Jacobians

The Jacobian J_x is the matrix of all first-order partial derivatives of a vector-valued function. For the prediction and measurement model, the Jacobians of the functions f and h are needed with respect to the state x (where $n = \text{length}(x)$). The Jacobian of any function with respect to x is defined as

$$J_x = \frac{\partial f}{\partial x} = \begin{bmatrix} \frac{\partial f_1}{\partial x_1} & \cdots & \frac{\partial f_1}{\partial x_n} \\ \vdots & \ddots & \vdots \\ \frac{\partial f_m}{\partial x_1} & \cdots & \frac{\partial f_m}{\partial x_n} \end{bmatrix} \tag{A.2.1}$$

Jacobians will be derived analytically in this section.

Partial derivatives of a function $\frac{\partial}{\partial Y} X(Y)$ are here denoted as X_Y. Please note that elements in vectors and matrices are indexed by w, x, y, z, these do not denote a partial derivative. The dynamics and measurement model Jacobians are pieced together from the Jacobians of the individual functions. The variable X is the output row, and the index $_Y$ denotes the (state) column. For example, the IMU measurement Jacobian is

$$H_x = \frac{\partial h_{IMU}}{\partial x} = \begin{bmatrix} 0_{3,4} & I_3 & 0_{3,3} & 0_{3,1} & 0_{3,1} & 0_{3,1} \\ M_q & 0_{3,3} & 0_{3,3} & 0_{3,1} & 0_{3,1} & 0_{3,1} \\ 0_{3,4} & 0_{3,3} & 0_{3,3} & P_l & 0_{3,1} & 0_{3,1} \end{bmatrix} \tag{A.2.2}$$

A.2.1 Kinematics and Quaternions

Quaternion Time Update

Recall: Quaternion mutliplication from Equation 1.10

$$p = q * r = \tilde{Q} \cdot r \qquad \tilde{Q} = \begin{bmatrix} q_w & -q_x & -q_y & -q_z \\ q_x & q_w & -q_z & q_y \\ q_y & q_z & q_w & -q_x \\ q_z & -q_y & q_x & q_w \end{bmatrix} \tag{A.2.3}$$

and the time derivative from Equation 1.11

$$\dot{q} = \frac{1}{2} q * \begin{bmatrix} 0 \\ \omega \end{bmatrix} = \frac{1}{2} \tilde{Q} \cdot \begin{bmatrix} 0 \\ \omega \end{bmatrix} = \frac{1}{2} \tilde{\Omega} \cdot q \qquad \tilde{\Omega} = \begin{bmatrix} 0 & -\omega_x & -\omega_y & -\omega_z \\ \omega_x & 0 & \omega_z & -\omega_y \\ \omega_y & -\omega_z & 0 & \omega_x \\ \omega_z & \omega_y & -\omega_x & 0 \end{bmatrix} \tag{A.2.4}$$

The Jacobian matrices for the quaternions time derivative are then

$$\dot{q}_q = \frac{1}{2}\tilde{\Omega} = \frac{1}{2}\begin{bmatrix} 0 & -\omega_x & -\omega_y & -\omega_z \\ \omega_x & 0 & \omega_z & -\omega_y \\ \omega_y & -\omega_z & 0 & \omega_x \\ \omega_z & \omega_y & -\omega_x & 0 \end{bmatrix} \tag{A.2.5}$$

$$\dot{q}_\omega = \frac{1}{2}\tilde{Q}_{:,2:4} = \frac{1}{2}\begin{bmatrix} -q_x & -q_y & -q_z \\ q_w & -q_z & q_y \\ q_z & q_w & -q_x \\ -q_y & q_x & q_w \end{bmatrix} \tag{A.2.6}$$

The discrete quaternion update from Equation 1.12

$$q(t+dt) = q(t) * dq(t, dt) \qquad \Longrightarrow \qquad q_{k+1} = \tilde{Q}_k \cdot dq_k \tag{A.2.7}$$

is approximated as

$$q(t+dt) = q(t) + \dot{q}\, dt \tag{A.2.8}$$

which results in the quaternion update Jacobians

$$q_q^+ = I_4 + \frac{1}{2}dt\,\tilde{\Omega} = \begin{bmatrix} 1 & -\omega_x\, dt/2 & -\omega_y\, dt/2 & -\omega_z\, dt/2 \\ \omega_x\, dt/2 & 1 & \omega_z\, dt/2 & -\omega_y\, dt/2 \\ \omega_y\, dt/2 & -\omega_z\, dt/2 & 1 & \omega_x\, dt/2 \\ \omega_z\, dt/2 & \omega_y\, dt/2 & -\omega_x\, dt/2 & 1 \end{bmatrix} \tag{A.2.9}$$

$$q_\omega^+ = \frac{1}{2}dt\,\tilde{Q}_{:,2:4} = \frac{1}{2}dt\begin{bmatrix} -q_x & -q_y & -q_z \\ q_w & -q_z & q_y \\ q_z & q_w & -q_x \\ -q_y & q_x & q_w \end{bmatrix} \tag{A.2.10}$$

Quaternion Vector rotations

The rotation operation R of a vector u by a quaternion q to a vector v (or from one frame to another frame) is

$$v = R(q, u) = Su = q * \begin{bmatrix} 0 \\ u \end{bmatrix} * q^{-1} \tag{A.2.11}$$

with the rotation matrix (see Equation 1.14)

$$S = 2q_v q_v^T + (q_w^2 - q_v^T q_v) I_3 - 2q_w \tilde{q}_v \tag{A.2.12}$$

$$= \begin{bmatrix} q_w^2 + q_x^2 - q_y^2 - q_z^2 & 2q_w q_z + 2q_x q_y & 2q_x q_z - 2q_w q_y \\ 2q_x q_y - 2q_w q_z & q_w^2 - q_x^2 + q_y^2 - q_z^2 & 2q_w q_x + 2q_y q_z \\ 2q_w q_y + 2q_x q_z & 2q_y q_z - 2q_w q_x & q_w^2 - q_x^2 - q_y^2 + q_z^2 \end{bmatrix} \tag{A.2.13}$$

The Jacobian of rotation after the vector u is just

$$R_u(q) = S \tag{A.2.14}$$

and the Jacobian of rotation after the quaternion q is

$$R_q(q, u) = 2\begin{bmatrix} q_w u - q_v \times u & q_v^T u\, I_3 + q_v u^T - u q_v^T + q_w \tilde{u} \end{bmatrix} \tag{A.2.15}$$

$$= 2\begin{bmatrix} q_w u_x - q_y u_z + q_z u_y & q_x u_x + q_y u_y + q_z u_z & q_x u_y - q_w u_z - q_y u_x & q_w u_y + q_x u_z - q_z u_x \\ q_w u_y + q_x u_z - q_z u_x & q_w u_z - q_x u_y + q_y u_x & q_x u_x + q_y u_y + q_z u_z & q_y u_z - q_w u_x - q_z u_y \\ q_w u_z - q_x u_y + q_y u_x & q_z u_x - q_x u_z - q_w u_y & q_w u_x - q_y u_z + q_z u_y & q_x u_x + q_y u_y + q_z u_z \end{bmatrix} \tag{A.2.16}$$

Relative motion

Position update in inertial frame from Equation 1.17 is

$$r^+ = r + dt\, S^T v \tag{A.2.17}$$

as $S^T = S(q^{-1})$, the position update Jacobians are

$$r_q^+ = dt\, R_q(q^{-1}, v) \tag{A.2.18}$$
$$r_v^+ = dt\, S^T \tag{A.2.19}$$
$$r_r^+ = I_3 \quad \implies l_l^+ = 1 \tag{A.2.20}$$

as the state is altitude (no the full 3D position), use just the first line of these matrices.

The cross product Jacobian is the skew symmetric matrix, so

$$(u \times v)_u = -\tilde{v} \qquad\qquad (u \times v)_v = \tilde{u} \tag{A.2.21}$$

with ($\tilde{v}$ would be the same structure, substitute u with v)

$$\tilde{u} = \begin{bmatrix} 0 & -u_z & u_y \\ u_z & 0 & -u_x \\ -u_y & u_x & 0 \end{bmatrix} \tag{A.2.22}$$

The velocity update in the body-fixed frame from Equation 1.21 is

$$v^+ = v + dt\,(a - \omega \times v + Sg^G) \tag{A.2.23}$$

which results in the Jacobians

$$v_q^+ = dt\, R(q, g^G) \tag{A.2.24}$$
$$v_\omega^+ = dt\, \tilde{v} \tag{A.2.25}$$
$$v_v^+ = I_3 - dt\, \tilde{\omega} \tag{A.2.26}$$
$$\tag{A.2.27}$$

The angular rate update in the body-fixed frame from Equation 1.18 is

$$\omega^+ = \omega + dt\, J^{-1}(\mathcal{T} - \omega \times J\omega) \tag{A.2.28}$$

when $\mathcal{T}$ is assumed to be constant w.r.t $\rho(l)$, this results in the Jacobians

$$\omega_\omega^+ = I_3 + dt\, J^{-1}(\mathcal{T}_\omega - (\tilde{J\omega})) \tag{A.2.29}$$
$$\omega_v^+ = dt\, J^{-1}\mathcal{T}_v \tag{A.2.30}$$
$$\omega_{C_L}^+ = dt\, J^{-1}\mathcal{T}_{C_L} \tag{A.2.31}$$
$$\omega_\delta^+ = dt\, J^{-1}\mathcal{T}_\delta \tag{A.2.32}$$

A.2.2 Sensor measurements

The sensors data is assumed to be already rotated into the body-fixed frame. The acceleration enters the filter via the dynamics input, and is thus not part of the formal measurement prediction.

The Jacobian of the gyroscope measurement is then

$$\Omega_\omega = I_3 \tag{A.2.33}$$

and the Jacobian of the magnetic field measurement is

$$M_q = R_q(q, M_E^G) \tag{A.2.34}$$

The pressure measurement has the Jacobian (derived in Section A.2.3)

$$P_l = p_l \tag{A.2.35}$$

and the canard angle encoder has the trivial Jacobian

$$\delta_{e\,\delta} = 1 \tag{A.2.36}$$

A.2.3 Atmosphere and air data

Partial derivatives of the US standard atmosphere, after altitude $X_l = \frac{\partial}{\partial l} X(l)$

$$\ell_l = \left(\frac{R_0}{R_0 - l}\right)^2 \tag{A.2.37}$$

$$T_l = -k_B\,\ell_l \tag{A.2.38}$$

$$p_l = p_B \ell_l \frac{g_0}{RT_B} \exp\left(\frac{-g_0}{RT_B}(\ell - \ell_B)\right) \;,\quad p_l = p_B \ell_l \frac{g_0}{RT_B}\left(1 - \frac{k_B}{T_B}(\ell - \ell_B)\right)^{\frac{g_0}{Rk_B}-1} \tag{A.2.39}$$

$$\rho_l = \frac{p_l T - p T_l}{R\,T^2} \tag{A.2.40}$$

$$c_l = \frac{1}{2} T_l \sqrt{\frac{\gamma R}{T}} \tag{A.2.41}$$

A.2.4 Aerodynamics

The dynamics need the partial derivatives of the external torque $\mathcal{T}$ from Equations 1.44, 1.47, 1.48

$$\mathcal{T} = \begin{bmatrix} \bar{p}\,C_{L_\delta}^{\text{can}}\,\delta A^{\text{can}} d_y^{\text{can}} \\ 0 \\ 0 \end{bmatrix} + \begin{bmatrix} \bar{p}\,C_{l_{\omega x}}^{\text{fin}}\,\omega_x A^{\text{fins}} d_y^{\text{fins}} \\ 0 \\ 0 \end{bmatrix} + \begin{bmatrix} 0 \\ \bar{p}\,C_{N_\alpha}\alpha A^{\text{ref}} d^{\text{ref}} \\ \bar{p}\,C_{N_\beta}\beta A^{\text{ref}} d^{\text{ref}} \end{bmatrix} \tag{A.2.42}$$

The density variation w.r.t the altitude is assumed to be very small, and therefore all $\mathcal{T}_l \approx \mathbf{0}$. Partial derivatives of the dynamic pressure $\bar{p}$, and the incidence angles α, β - pressure product are

$$\bar{p}_v = \rho \begin{bmatrix} v_x & v_y & v_z \end{bmatrix} \tag{A.2.43}$$

$$(\bar{p} \begin{bmatrix} \alpha \\ \beta \end{bmatrix})_v \approx \frac{\rho}{2} \begin{bmatrix} v_z & 0 & v_x + 3v_z^2/v_x \\ -v_y & -v_x - 3v_y^2/v_x & 0 \end{bmatrix} \tag{A.2.44}$$

$$\approx \frac{\rho}{2} \begin{bmatrix} v_z & 0 & v_x \\ -v_y & -v_x & 0 \end{bmatrix} \tag{A.2.45}$$

The Jacobians of the external torques are then

$$\mathcal{T}_\omega = \begin{bmatrix} \bar{p}\, C_{l_{\omega_x}}^{\text{fin}} A^{\text{fins}} d_y^{\text{fins}} & 0 & 0 \\ 0 & 0 & 0 \\ 0 & 0 & 0 \end{bmatrix} \approx \mathbf{0} \tag{A.2.46}$$

$$\mathcal{T}_v = \begin{bmatrix} C_L \delta A^{\text{can}} d_y^{\text{can}} \\ 0 \\ 0 \end{bmatrix} \bar{p}_v \tag{A.2.47}$$

$$+ \frac{\rho}{2} C_{N_\alpha} A^{\text{ref}} d^{\text{ref}} \begin{bmatrix} 0 & 0 & 0 \\ v_z & 0 & v_x \\ -v_y & -v_x & 0 \end{bmatrix} \tag{A.2.48}$$

$$\mathcal{T}_{C_L} = \begin{bmatrix} \bar{p}\, \delta A^{\text{can}} d_y^{\text{can}} \\ 0 \\ 0 \end{bmatrix} \tag{A.2.49}$$

$$\mathcal{T}_\delta = \begin{bmatrix} \bar{p}\, C_L A^{\text{can}} d_y^{\text{can}} \\ 0 \\ 0 \end{bmatrix} \tag{A.2.50}$$

A.2.5 Actuator

The actuator update is

$$\delta^+ = \delta + dt\, \alpha_\delta (\delta_u - \delta) \tag{A.2.51}$$

and the Jacobian is then

$$\delta_\delta = 1 - dt\, \alpha_\delta \tag{A.2.52}$$

A.2.6 Airfoil

The airfoil estimate is

$$C_L^+ = C_L + dt\, \alpha_c (C_{L,\,\text{theory}} - C_L) \tag{A.2.53}$$

With the theoretical value from Equations 1.42 and 1.43, and $\mu = \arccos(\frac{1}{\text{Ma}})$

$$\text{Ma} \leq 1: \quad C_{L,\,\text{theory}} = 2\pi \cot(\Lambda) \tag{A.2.54}$$

$$\mu \leq \Lambda: \quad C_{L,\,\text{theory}} = \frac{2\pi^2 \cot(\Lambda)}{\pi + \eta(0.38 + 2.26\eta - 0.86\eta^2)} \quad \text{with} \quad \eta = \frac{\cot(\Lambda)}{\cot(\mu)} \tag{A.2.55}$$

$$\mu > \Lambda: \quad C_{L,\,\text{theory}} = \frac{4}{\sqrt{\text{Ma}^2 - 1}} \tag{A.2.56}$$

As the theoretical value is very cumbersome to partially differentiate, it is practical to assume it linear respective all variables. The curve goes through $C_L(\mathrm{Ma}=1)$, and $C_L(\mu = \Lambda)$ which corresponds to the supercritical Mach number, where the shock wave angle is equal or greater than the canard leading edge. The supercritical point is $\mathrm{Ma} = \frac{1}{\cos(\Lambda)}$, and the linear approximation is (shown in Figure A.2.1)

$$\mathrm{Ma} \leq 1: \qquad C_{L,\,\mathrm{theory}} = 2\pi\cot(\Lambda) \tag{A.2.57}$$

$$\mathrm{Ma} > 1: \qquad C_{L,\,\mathrm{theory}} = 2\pi\cot(\Lambda) + \frac{(2\pi - 4.01)\cot(\Lambda)}{1 - 1/\cos(\Lambda)}(\mathrm{Ma} - 1) \tag{A.2.58}$$

This results in the (clean) Jacobians

$$C_{L\,C_L} = 1 - dt\,\alpha_c \tag{A.2.59}$$

$$C_{L\,v} = 0 \quad \text{for} \quad \mathrm{Ma} \leq 1 \tag{A.2.60}$$

$$C_{L\,v} = dt\,\alpha_c \underbrace{\frac{(2\pi - 4.01)\cot(\Lambda)}{1 - 1/\cos(\Lambda)}}_{C_{L\ \mathrm{slope}}} \cdot \frac{1}{c} \quad \text{for} \quad \mathrm{Ma} > 1 \tag{A.2.61}$$

$$C_{L\,l} \approx 0 \tag{A.2.62}$$

where the factor $C_{L\ \mathrm{slope}}$ can be pre-computed.

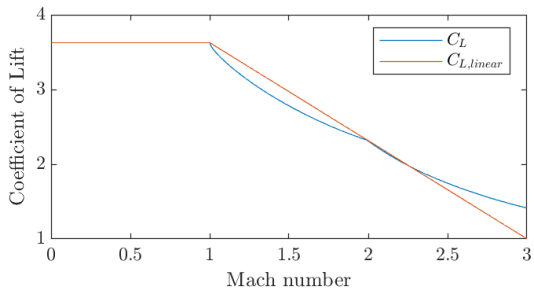


Figure A.2.1: Canard airfoil approximation